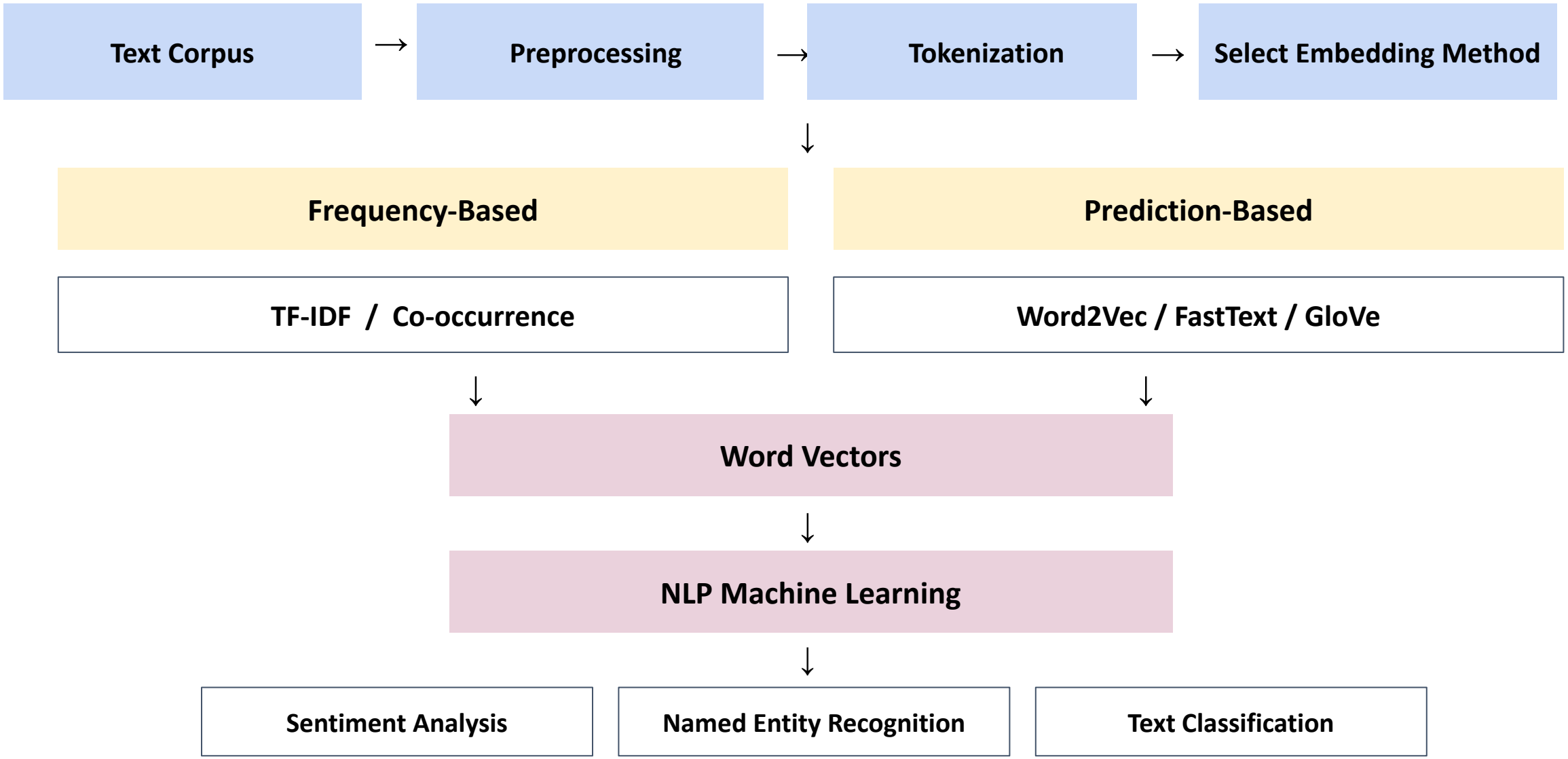
UNIT 3

Introduction to Word Embeddings

3	**Introduction to Word Embeddings:** Word Embeddings, and Word Vectors (word2Vec), Gensim wordvectors example, Word Vectors 2 and Word Senses (GloVe), Word Window Classification, Neural Networks, Matrix Calculus and Backpropagation, Linguistic Structure: Dependency Parsing, Negative Sampling.

Overall Word Embedding Flow



What Is Word Embedding?

A technique in NLP that represents words as numerical vectors capturing meaning and relationships.

Computers cannot read words

`king, queen, doctor, teacher`

So we convert words into vectors of numbers that capture meaning:

King	`[0.75, 0.82, 0.21, 0.65]`
Queen	`[0.73, 0.80, 0.25, 0.68]`
Apple	`[0.12, 0.20, 0.85, 0.10]`

Similar meanings → similar vectors

Cat *similar* **Dog**

Both are animals

Numerical Vector → Semantic Relationship + Syntactic Relationship

Why Do We Need Word Embeddings?

Traditional (One-Hot) Representation

```
"cat"   →   [1, 0, 0, 0]
"dog"   →   [0, 1, 0, 0]
"car"   →   [0, 0, 1, 0]
"book"  →   [0, 0, 0, 1]
```

Tells us the words are different — but not how they relate.

✕ **No sense of similarity or relationship**

Word Embeddings Solve This

≈ Similar Meaning → Numerical Vector

Semantic Relationship

Syntactic Relationship

Example: Cat ——similar—— Dog (both are animals)

Importance of Word Embeddings

1 Semantic Representation

Word embeddings help machines understand meaning — similar words sit close together in vector space.

King ≈ Queen

Car ≈ Vehicle

Happy ≈ Joyful

2 Lower-Dimensional Representation

One-hot encoding can need thousands of dimensions; embeddings compress the same words into far fewer.

One-Hot Encoding

`Word → 10,000 dimensions`

↓ more efficient ↓

Word Embedding

`Word → 100 or 300 dimensions`

Importance of Word Embeddings

3 Captures Relationships

Embeddings learn relationships between words directly from large amounts of text.

```
King – Man + Woman ≈ Queen
```

A famous example of vector arithmetic capturing analogy.

Transfer Learning

Pre-trained embeddings can be reused for new NLP tasks — especially useful when training data is small.

4 Useful for NLP Tasks

Sentiment Analysis

Named Entity Recognition

Machine Translation

Text Classification

Question Answering

Recommendation Systems

Main Types of Word Representation

Frequency-Based Methods

Rely on how often words occur in documents or alongside other words.

TF-IDF

Co-occurrence Matrix

Prediction-Based Methods

Learn word vectors by predicting words from their surrounding context.

Word2Vec (CBOW · Skip-gram)

FastText

GloVe

Word2Vec

Word2Vec changed NLP by representing words as **dense numerical vectors**, helping computers understand the **meaning and relationships between words**.

Dense numerical vectors mean:

A word is represented by a list of numbers, where most of the numbers contain useful information

History of Word2Vec

2013

Introduced by a Google research team led by Tomas Mikolov

1

Two Models Released

The team introduced two architectures: the **Continuous Bag of Words (CBOW) model and the Skip-gram model.**

2

Built at Google

Developed as part of Google’s research into representing language numerically for machine learning.

3

Widely Adopted Since

Word2Vec saw rapid, wide adoption across industries, driven by its effectiveness at capturing semantic relationships between words.

Functionality and Features

What Word2Vec actually does with a text corpus

1

Converts Text to Numbers

Turns raw text into a numerical form that machine learning algorithms generally — can process.

2

Learned via Deep Learning

Uses deep learning, especially neural networks, to map semantic meaning into a geometric embedding space.

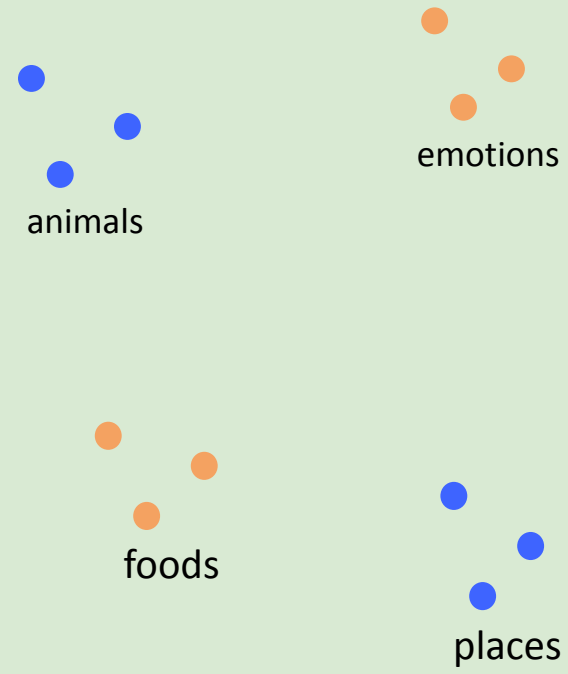
3

Groups Similar Meaning

Words that share contextual meaning end up positioned close together within that embedding space.

Embedding Space

Similar-meaning words cluster together



Word2Vec (EXAMPLE)

Suppose we have these sentences:

- **The cat drinks milk.**
- **The dog drinks milk.**

Word2Vec learns that **“cat”** and **“dog”** are related because they appear in similar contexts.

It represents words as numbers (vectors), for example:

- **cat → [0.2, 0.8, 0.3]**
- **dog → [0.3, 0.7, 0.4]**
- **milk → [0.9, 0.1, 0.2]**

Since **cat** and **dog** have similar vectors, Word2Vec understands that they have **similar meanings or are related**.

Each word has 3 numbers, so we say it is a 3-dimensional vector

cat → [0.2, 0.8, 0.3]

dog → [0.3, 0.7, 0.4]

We cannot say that Dimension 1 specifically means "animal" or Dimension 2 means "food". Word2Vec learns these numbers automatically from the text.

- Word2Vec notices that 'cat' and 'dog' occur in similar surroundings.
- So it learns vector values that make related words have similar vectors.

But in real Word2Vec?

The vector is usually much larger than 3 dimensions, such as:

cat ⟶ [0.12, -0.45, 0.78, 0.21, ...]

↑

100–300 numbers

So 3 dimensions are easy to understand and visualize.

The number of dimensions is chosen as a model setting (e.g., 100, 200, 300), **and Word2Vec learns the values from the training text.**

Architecture Flow

A simple two-layer neural network produces the word vectors

Input Layer

Accepts the encoded (one-hot) representation of each word.

→

Hidden Layer

Computes the word vectors based on surrounding context.

→

Output Layer

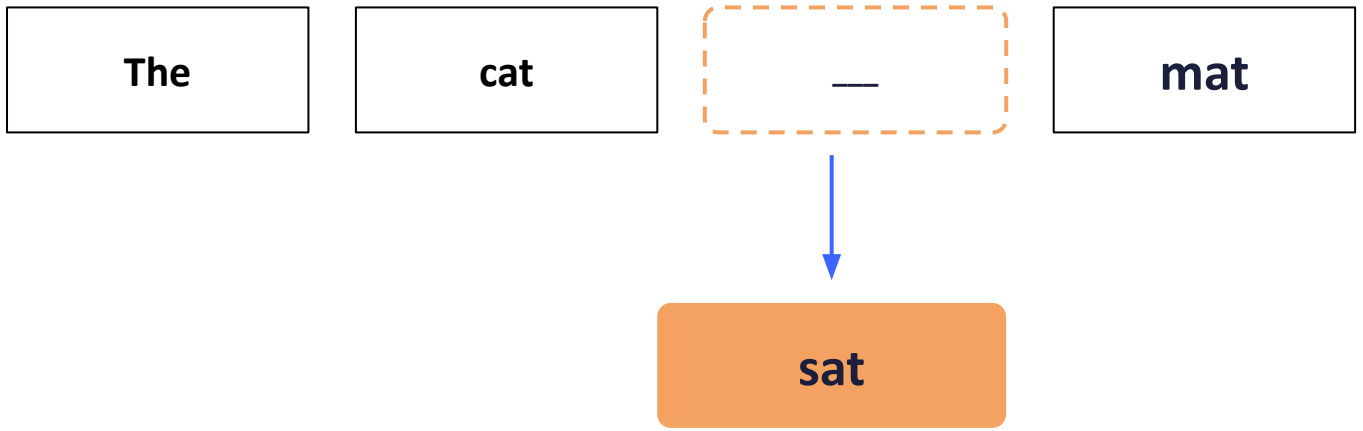
Produces the final word vectors (embeddings).

Word2Vec — Two Architectures

A neural approach that learns embeddings by predicting words from their context

CBOW

Continuous Bag of Words

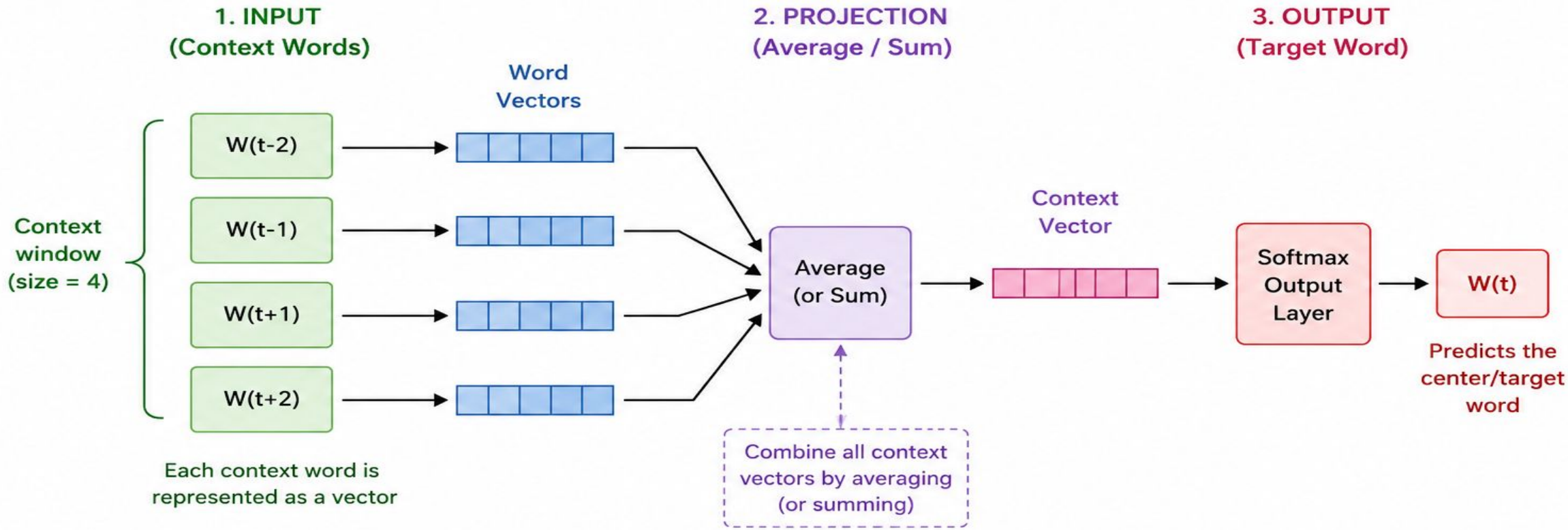


context words → predict target word

- Uses surrounding context words to predict the missing target word.
- Faster to train; works well for frequent words.

CBOW (Continuous Bag of Words) Model

CBOW predicts the target (center) word using the surrounding context words.



Example:

Sentence: "The cat sits on mat"

"The	cat	sits	on	mat"
W(t-2)	W(t-1)	W(t)	W(t+1)	W(t+2)

Context Words: The, cat, on, the
(used to predict)

Target Word: **sits**

CBOW (Continuous Bag of Words)

Context Words as One-Hot Vectors

CBOW predicts the target (center) word from its surrounding context words.

Example Sentence

Cat	sat	on	mat
1	2	3	4

Target (center) word

Window Size = 1 (for word "sat")

Context words: "cat" and "on"

cat	sat	on

Context Words as One-Hot Vectors

Vocabulary = { **cat, sat, on, mat** } (V = 4)

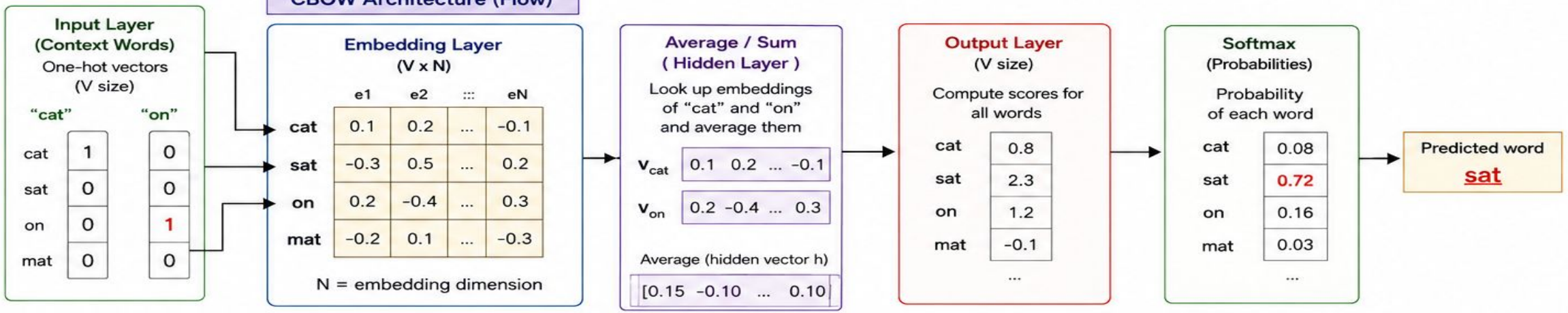
One-hot for "cat"

Word	One-Hot Vector (V=4)
cat	[1 0 0 0]
sat	[0 1 0 0]
on	[0 0 1 0]
mat	[0 0 0 1]

One-hot for "on"

Word	One-Hot Vector (V=4)
cat	[0 0 0 0]
sat	[0 1 0 0]
on	[0 0 1 0]
mat	[0 0 0 1]

CBOW Architecture (Flow)



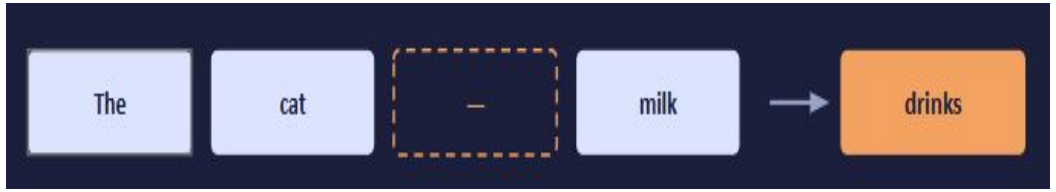
How it works for (Context: cat, on → Target: sat)

1. Convert each context word ("cat" and "on") into one-hot vectors of size V=4.
2. Use these one-hot vectors to look up their embeddings from the embedding matrix.
3. Average (or sum) the embeddings to get the hidden vector h.
4. Multiply h with the output weight matrix to get a score for every word in the vocabulary.
5. Apply softmax to get probabilities. The highest probability word is the predicted target word "sat".

Key Points

- One-hot vector has 1 at the position of the word and 0 elsewhere.
- CBOW uses multiple context words (as one-hot vectors), looks up their embeddings, averages them, and predicts the center word.

Step 1 & 2 — From Words to Vectors



Example sentence: “The cat drinks milk” — predicting “drinks”

Input — Context Words

The surrounding words are given to the model as input.

The **cat** **milk**

target word to predict: “drinks”

These three words form the “context” that CBOW will use to guess the missing word in the middle.

Embedding Layer

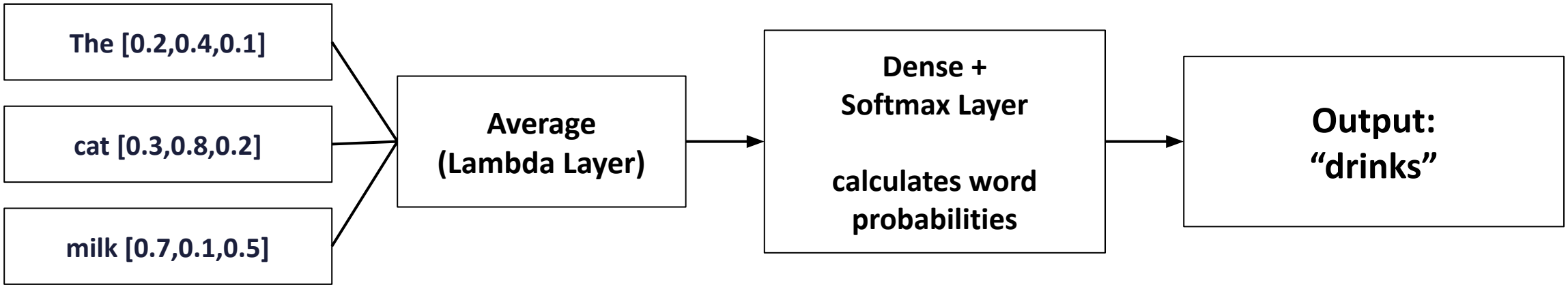
Each word is converted into a vector of numbers.

The	`[0.2, 0.4, 0.1]`
cat	`[0.3, 0.8, 0.2]`
milk	`[0.7, 0.1, 0.5]`

These vectors capture meaning — similar words get similar numbers.

Step 3–5—From Context to Prediction

How the averaged context vector becomes a predicted word



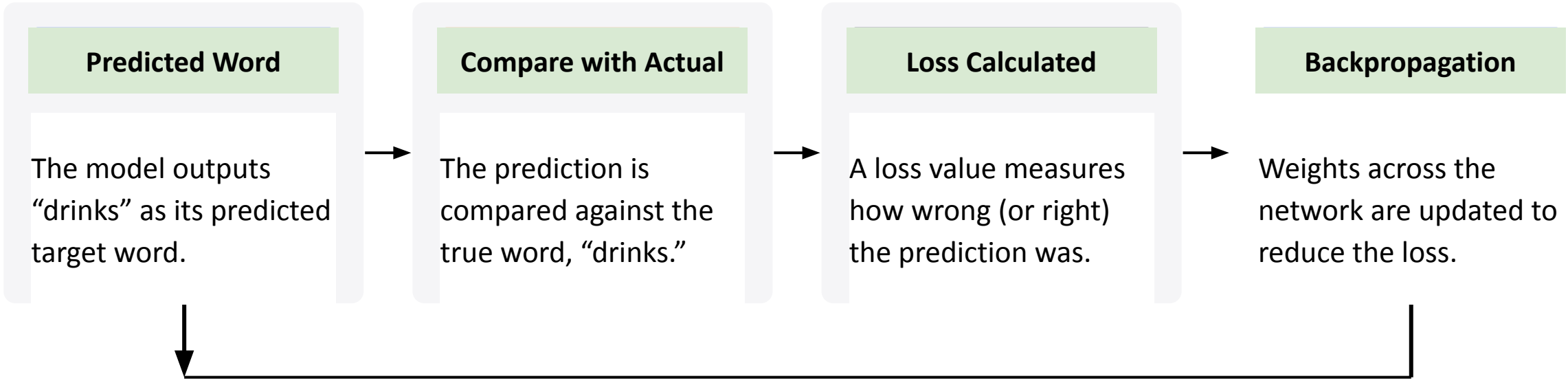
The cat and milk vectors are combined into a single averaged vector, which is then passed through a Dense + Softmax layer that scores every word in the vocabulary.

Softmax ranks every candidate word by probability



Step 6 — Training the Model

How CBOW learns better embeddings over time

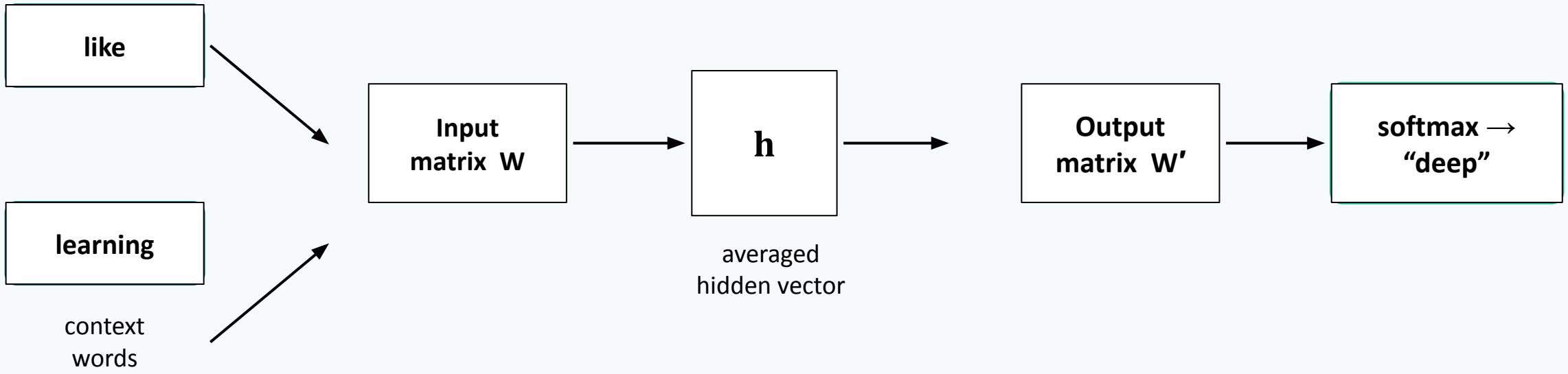


The Result

After many rounds of this cycle, the embedding vectors settle into values where words used in similar contexts end up close together in vector space.

CBOW: Numerical

How CBOW Predicts a Word



1 · Combine

All context word vectors are looked up in W and averaged into a single hidden vector h.

2 · Score

h is compared against every word's output vector in W′ via a dot product to get raw scores.

3 · Predict

Softmax turns scores into probabilities; the model is trained to push probability mass onto the true target word.

STEP 0

The Sentence, Vocabulary & Embeddings

I	like	deep	learning

window size = 1 ⟶ target “deep” · context = { like, learning }

Input embeddings W (looked up for context words)

I	like	deep	learning
[0.1, 0.2]	[0.4, 0.3]	[0.2, 0.5]	[0.6, 0.1]

Output embeddings W′ (one vector per vocabulary word)

I	like	deep	learning
[0.1, 0.3]	[0.2, 0.4]	[0.4, 0.1]	[0.3, 0.2]

Given

- Vocabulary size V = 4
- Embedding dim = 2
- Target word: “deep”
- Context words: “like”, “learning”
- Learning rate η = 0.1

Why do we use two matrices: W and W′?

In Word2Vec, W and W′ have different roles.

- W → used when a word is a context/input word.
- W′ → used when a word is the target/output word being predicted.

Why not use one matrix?

Using separate matrices allows the model to learn two different roles:

W: “How does this word help predict another word?”
W′: “How well can this word be predicted?”

During training, both W and W′ are updated using gradient descent.

How are W and W′ values obtained?

1. Before training – Random Initialization

The values in W and W′ are initially chosen randomly.

For example:

- W → [0.1, 0.2]
- W′ → [0.4, 0.3]

These are just starting values. They are not calculated from the data.

2. During training – Values are Updated

The model learns by repeatedly:

Forward Pass → Calculate Error → Backpropagation → Update W and W′

The values are updated using gradient descent.

For example:

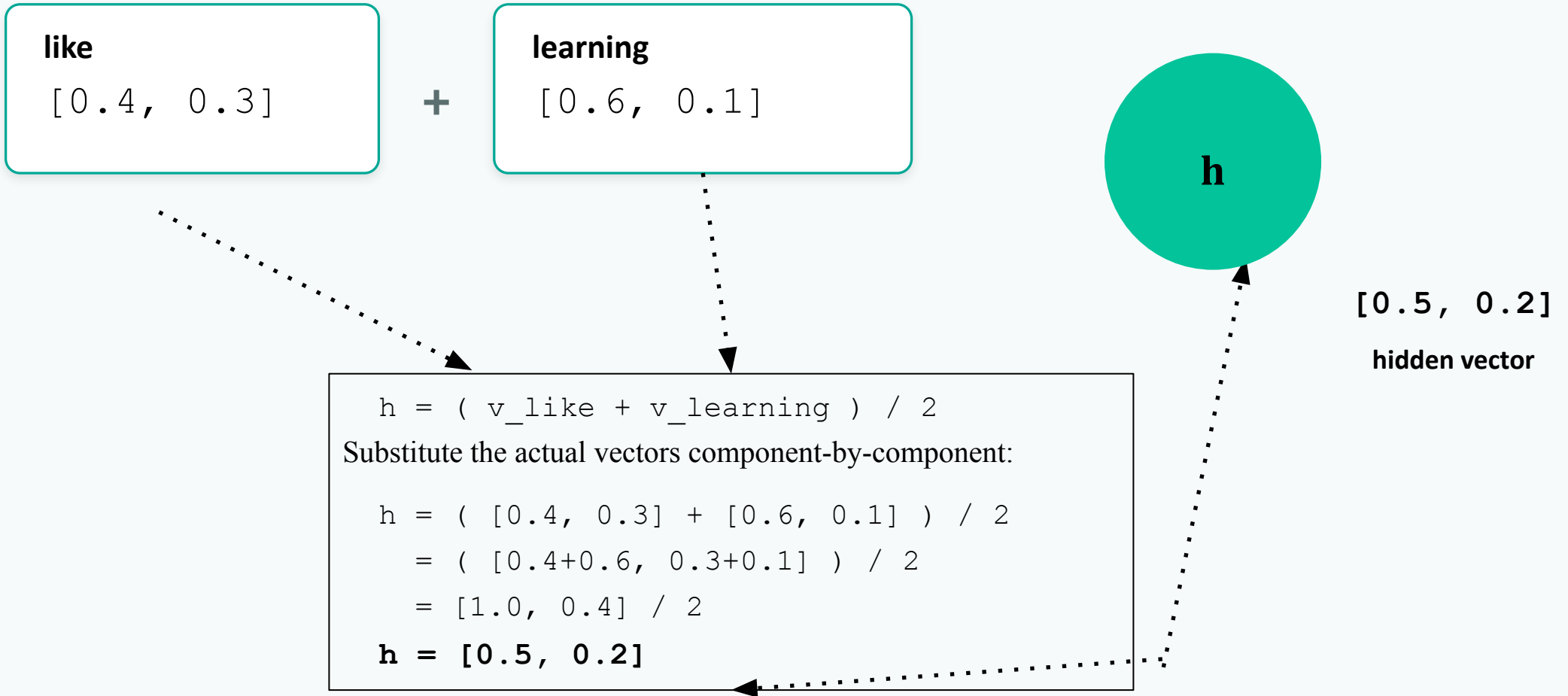
Old value → Calculate error → Small update → New value

0.4 → error → +0.0369 → 0.4369

The process is repeated many times for different training examples.

Average the Context Vectors → Hidden Layer h

Only “like” and “learning” are looked up from W in this pass — they are the context words. Every word's row in W′ gets used, because softmax scores the entire vocabulary as candidates.



Step 2 · Forward Pass — Score Every Word, Then Apply Softmax

Next, h is compared against every word's output vector (from W′) using a dot product. This produces a raw “score” for each vocabulary word:

score(w) = h · u^{W}

- score(I) = (0.5)(0.1) + (0.2)(0.3) = 0.05 + 0.06 = 0.11
- score(like) = (0.5)(0.2) + (0.2)(0.4) = 0.10 + 0.08 = 0.18
- score(deep) = (0.5)(0.4) + (0.2)(0.1) = 0.20 + 0.02 = 0.22
- score(learning) = (0.5)(0.3) + (0.2)(0.2) = 0.15 + 0.04 = 0.19

Softmax turns these raw scores into probabilities that sum to 1. First exponentiate every score:

e^0.11 ≈ 1.116 e^0.18 ≈ 1.197 e^0.22 ≈ 1.246 e^0.19 ≈ 1.209

Sum them to get the softmax denominator:

Σ e^score = 1.116 + 1.197 + 1.246 + 1.209 = 4.768

Then divide each word's e^score by this sum:

P(w) = e^score(w) / 4.768

After applying softmax

score(w) = h · u□

P(w | context) = e^score(w) / Σ e^score(w')

★ = the true context word for this training pair.

Sanity check: 23.4% + 25.1% + 26.1% + 25.4% = 100%, exactly as softmax requires.

The model currently gives “deep” only 26.1% confidence the goal of training is to push this toward 100%.

Word	u□	score = h·u□	e^score	P(w)
I	[0.1, 0.3]	0.11	1.116	23.4%
like	[0.2, 0.4]	0.18	1.197	25.1%
deep ★	**[0.4, 0.1]**	**0.22**	**1.246**	**26.1%**
learning	[0.3, 0.2]	0.19	1.209	25.4%

STEP 3 · LOSS

Cross-Entropy Loss for the True Word

$$L = -\log P(\text{deep})$$

$$= -\log(0.2613)$$

$$\approx 1.342$$

Higher loss = the model was less confident about the correct word.

The sign of e^W tells the model which direction to move each output vector: negative error (deep) means “pull this vector closer to h”; positive error (everyone else) means “push this vector away from h.”

Error signal $e_□ = P(w) - y(w)$ (y = 1 only for the true word)

I	0.234 − 0 = +0.234	push away
like	0.251 − 0 = +0.251	push away
deep ★	0.261 − 1 = −0.739	pull closer
learning	0.254 − 0 = +0.254	push away

Step 4 · Backward Pass — Update the Weights

First, compute how the loss changes with respect to h. This sums every word's error, weighted by that word's output vector:

$$\partial L/\partial h = \Sigma^{W} e^{W} \cdot u^{W}$$

Expanding term by term:

- $e_i \cdot u_i = 0.234 \times [0.1, 0.3] = [0.0234, 0.0702]$
- $e_like \cdot u_like = 0.251 \times [0.2, 0.4] = [0.0502, 0.1004]$
- $e_deep \cdot u_deep = -0.739 \times [0.4, 0.1] = [-0.2956, -0.0739]$
- $e_learning \cdot u_learning = 0.254 \times [0.3, 0.2] = [0.0762, 0.0508]$

Summing the x-components: $0.0234 + 0.0502 - 0.2956 + 0.0762 = -0.1458$

Summing the y-components: $0.0702 + 0.1004 - 0.0739 + 0.0508 = 0.1475$

$$\partial L/\partial h \approx [-0.146, 0.147]$$

Step 4: Propagate the Error Back to Context Words

Because h was built by averaging 2 context vectors, the chain rule splits this gradient evenly between the two context words:

grad per context word = $(\partial L/\partial h) \div 2$

grad per context word = [−0.073, 0.074]

Finally, **apply gradient descent** — each vector moves a small step (scaled by learning rate $\eta = 0.1$) in the direction that reduces loss:

v_new = v_old − η × grad

v_like: [0.4, 0.3] − 0.1 × [−0.073, 0.074] = [0.4+0.0073, 0.3−0.0074] = **[0.407, 0.293]**

v_learning: [0.6, 0.1] − 0.1 × [−0.073, 0.074] = [0.6+0.0073, 0.1−0.0074] = **[0.607, 0.093]**

One CBOW Training Step, End to End

1

Average

Look up context word vectors in W and average them into hidden vector h.

2

Score & Softmax

Compare h against every word's output vector; softmax turns scores into probabilities.

3

Loss

Cross-entropy loss compares predicted P(target) against the ideal value of 1.

4

Update

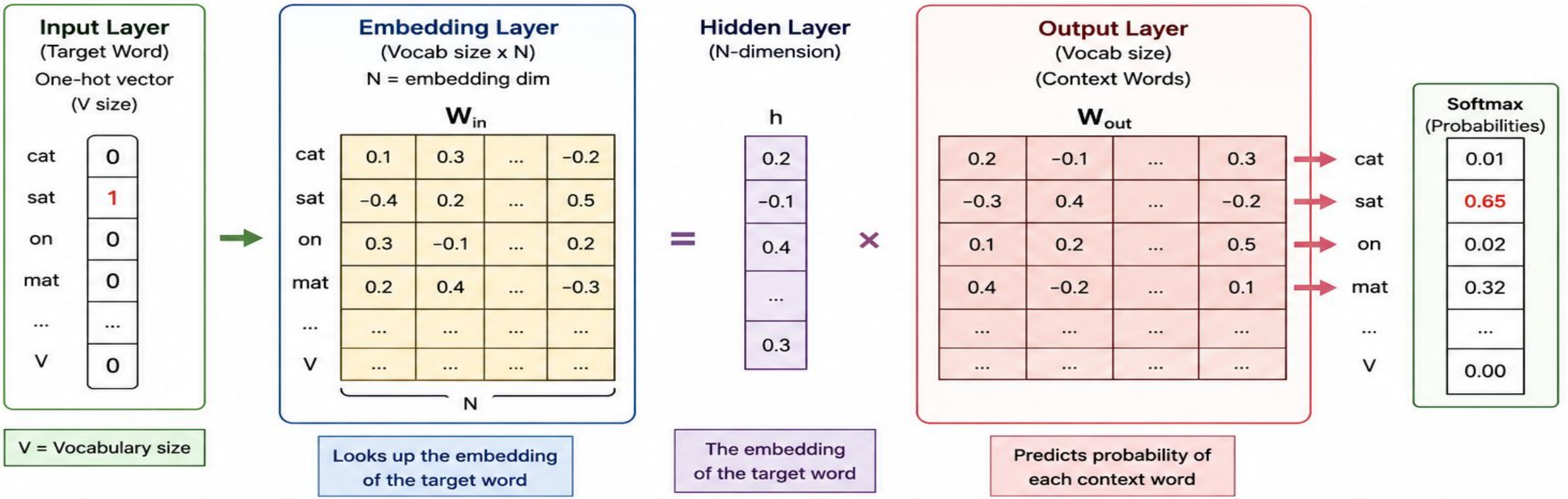
The error signal flows back through W′ and W, nudging both toward better predictions.

Repeat across millions of context windows and the embeddings converge — semantically similar words end up with similar vectors.

Skip-gram

Predicting Context Words from a Single Target Word

Skip-Gram Architecture



Example Sentence

Cat sat on mat

1 2 3 4

Vocabulary

{ cat, sat, on, mat }

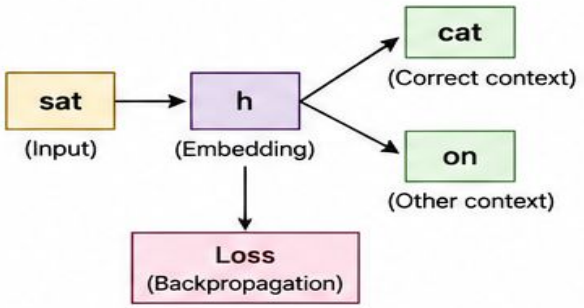
V = 4

Skip-Gram with Window Size = 1

Target Word	Context Words	(Input → Output Pairs)
cat	sat	(cat → sat)
sat	cat, on	(sat → cat), (sat → on)
on	sat, mat	(on → sat), (on → mat)
mat	on	(mat → on)

How it works for one pair: (sat → cat)

1. **Target word = sat**
   → One-hot vector for "sat"
2. **Lookup embedding of "sat"**
   → Get hidden vector h
3. **Predict context words**
   → Compute scores for all words
4. **Softmax**
   → Higher probability for "cat"
5. **Update weights**
   → Using loss (e.g., cross-entropy)



How Skip-gram Works

Given a target (centre) word, Skip-gram predicts the surrounding context words within a fixed window — the reverse of CBOW.



window size = 2 → 2 words of context on each side of the target word are used

Target word: “drinks” → predicts context words: “The”, “cat”, “milk”, “quickly”

One Word In

The input layer holds a single target word, W(t) — e.g. “drinks.”

Many Words Out

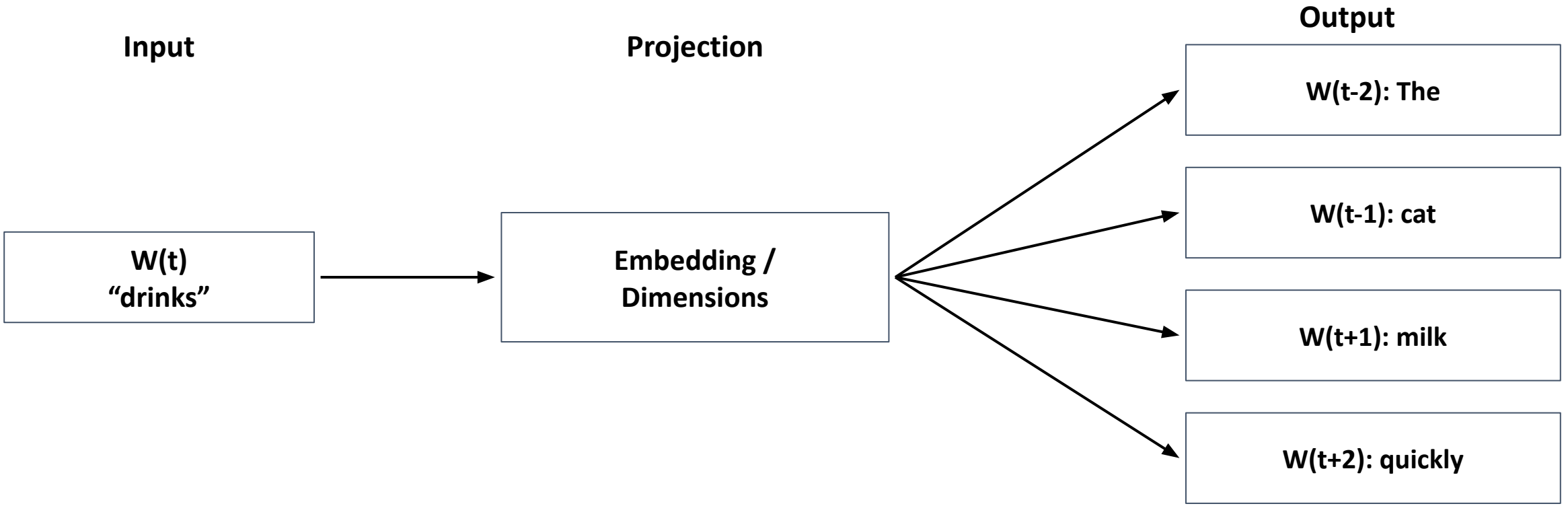
The output layer holds several context word positions: W(t−2), W(t−1), W(t+1), W(t+2).

Trained in Pairs

Word2Vec turns this into (target, context) training pairs fed to the network one at a time.

Architecture 1 — Softmax Skip-gram

Input ⟶ Projection ⟶ Output, predicting each context position directly



The projection layer size sets the embedding dimensionality.

A separate Softmax prediction is made for every context position, and the model is trained to maximise the probability of the true context word at each position.

Positive & Negative Sampling

it asks a much simpler yes/no question, repeated many times

How Skip-gram turns “one word predicts many” into simple (X, Y) training pairs

Instead of predicting a full context at once, each (target, other word) pair becomes its own training example, labelled 1 if the words are real neighbours and 0 if the word was chosen at random.

Positive Samples

real (target, context) pairs ⟶ label 1

```
(drinks, cat)   →   1
(drinks, milk)   →   1
(drinks, The)   →   1
```

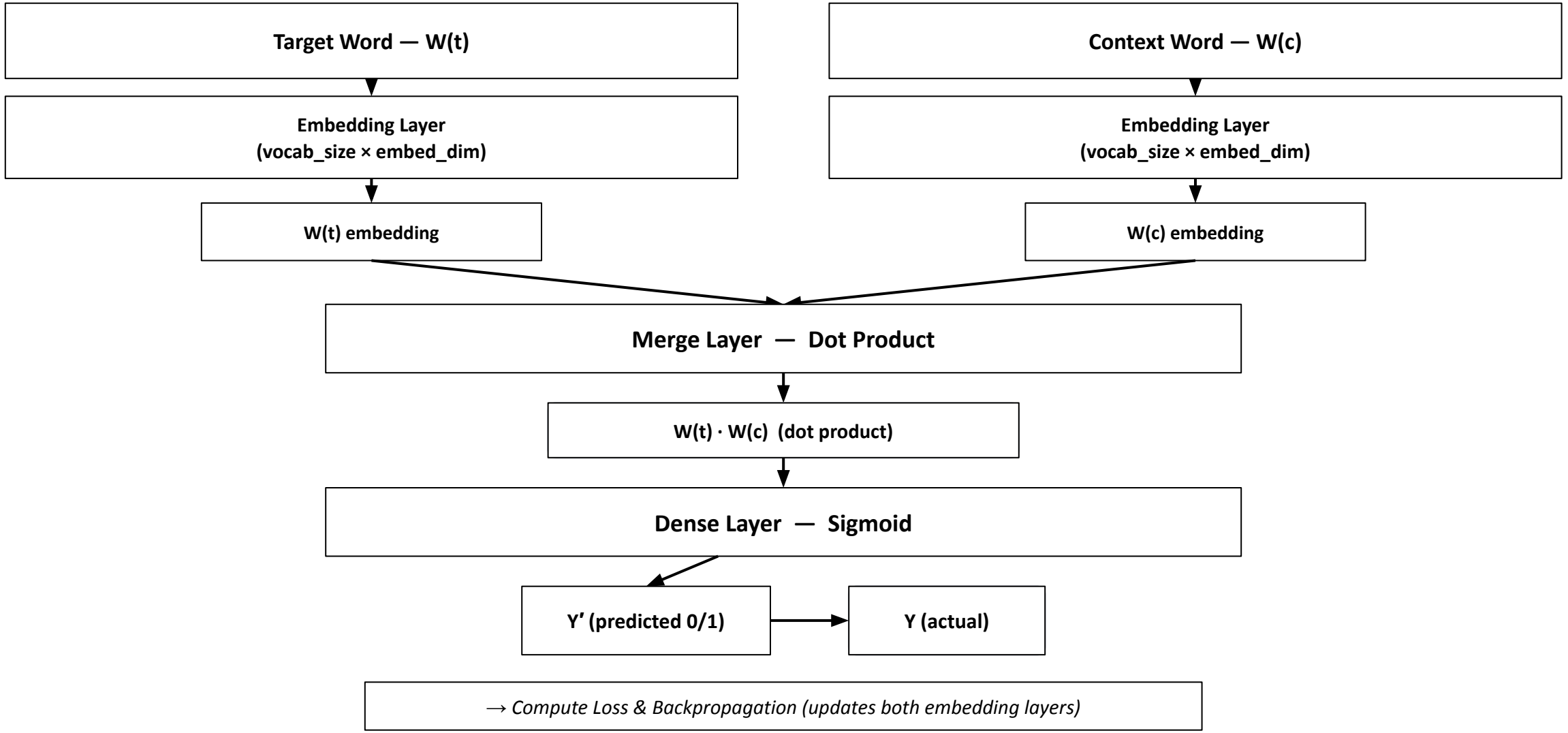
Negative Samples

(target, random word) pairs ⟶ label 0

```
(drinks, umbrella)   →   0
(drinks, Paris)   →   0
(drinks, algebra)   →   0
```

Architecture 2 — Skip-gram with Negative Sampling

Two embeddings, a dot product, and a sigmoid classifier — far cheaper to train than full Softmax



Why is it called Negative Sampling?

In Skip-gram, the positive pair already comes from the sentence.

Example:

“sat” → “cat” Positive pair → 1

Then we randomly choose wrong words:

“sat” → “dog” → 0

“sat” → “car” → 0

So:

Negative Sampling = 1 real (positive) pair + a few randomly selected negative pairs.

It is called negative sampling because we sample the incorrect/negative words to make training faster.

Skip-gram: Numerical

Using the sentence “I like deep learning”

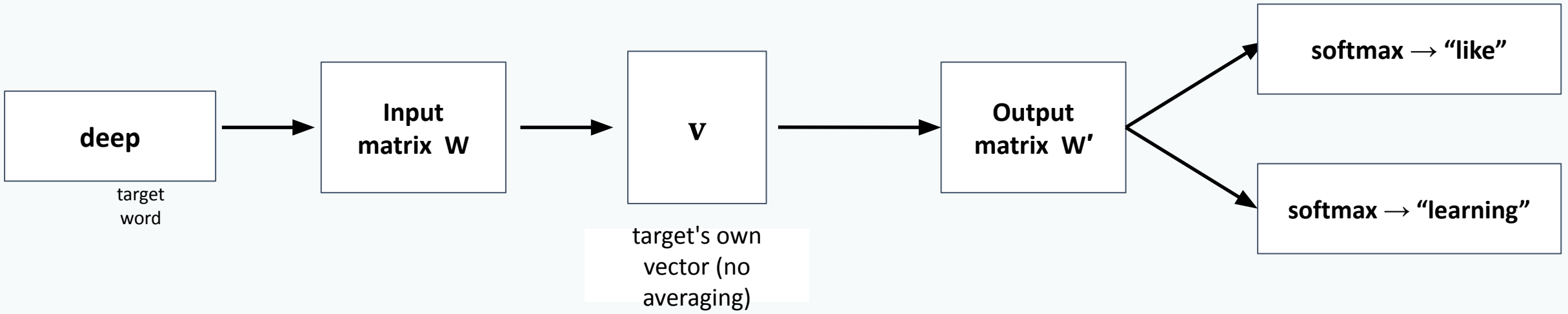
- window size = 1
- target word “deep”
- embedding dimension = 2
- learning rate $\eta = 0.1$.

The target “deep” generates two training pairs:

- (deep, like)
- (deep, learning).

It follows the pair **(deep → like)**

How Skip-gram Predicts Context



1 · Look Up

The target word's own vector is taken directly from W—there is no averaging, unlike CBOW.

2 · Score

That single vector is compared against every word's output vector in W′ via a dot product.

3 · Predict, Per Pair

Each context word forms its own training pair; softmax is run once per pair to predict it.

STEP 0

The Sentence, Vocabulary & Training Pairs

I	like	deep	learning

window size = 1 → target “deep” generates 2 training pairs: (deep, like) and (deep, learning)

Input embeddings W (looked up for the target word)

I	like	deep	learning
[0.1, 0.2]	[0.4, 0.3]	[0.2, 0.5]	[0.6, 0.1]

an input/target vector (row in W)

Output embeddings W’ (one vector per vocabulary word)

I	like	deep	learning
[0.1, 0.3]	[0.2, 0.4]	[0.4, 0.1]	[0.3, 0.2]

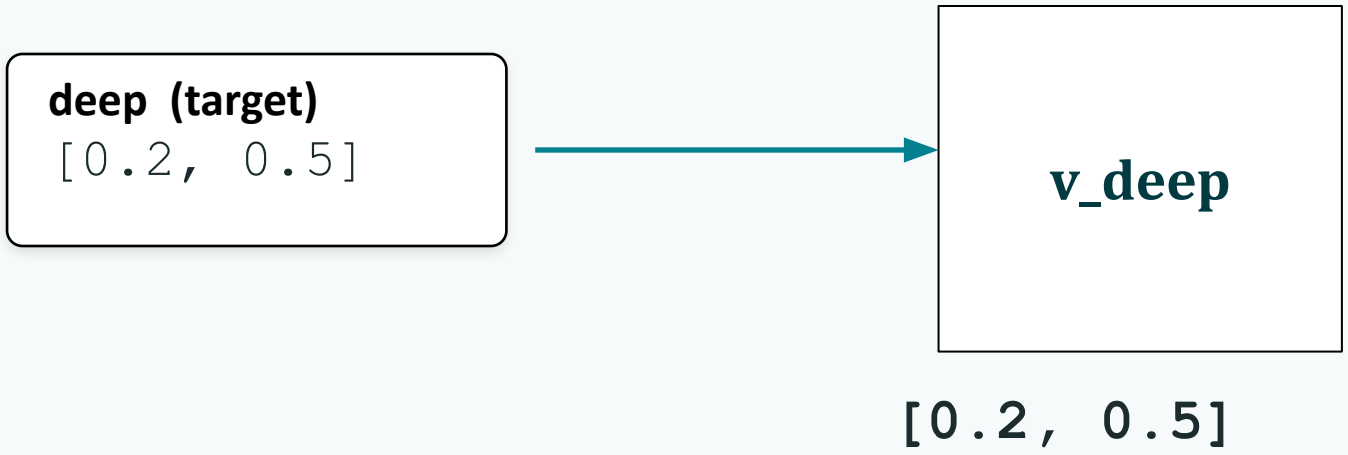
an output/context vector (row in W′).

Given

- Vocabulary size V = 4
- Embedding dim = 2
- Target word: “deep”
- This walkthrough: pair (deep, like)
- Learning rate η = 0.1

STEP 1 · FORWARD PASS

Look Up the Target's Own Vector — No Averaging



Skip-gram skips CBOW's averaging step entirely. Because there is only one input word, the hidden vector h is simply the target word's own input embedding:

This single vector is now reused, as-is, for scoring every context prediction — for both the (deep, like) and (deep, learning) pairs.

Step 2 · Forward Pass — Score Every Word, Then Apply Softmax

h is compared against every word's output vector (from W′) using a dot product:

score(w) = $v_{deep} \cdot u^{W}$

- score(I) = (0.2)(0.1) + (0.5)(0.3) = 0.02 + 0.15 = 0.17
- score(like) = (0.2)(0.2) + (0.5)(0.4) = 0.04 + 0.20 = 0.24
- score(deep) = (0.2)(0.4) + (0.5)(0.1) = 0.08 + 0.05 = 0.13
- score(learning) = (0.2)(0.3) + (0.5)(0.2) = 0.06 + 0.10 = 0.16

Exponentiate each score, sum them, then divide — this is softmax:

e^0.17≈1.185 e^0.24≈1.271 e^0.13≈1.139 e^0.16≈1.174

∑ e^score = 1.185 + 1.271 + 1.139 + 1.174 = **4.768**

STEP 2 · FORWARD PASS

Word	u□	score = v·u□	e^score	P(w)
I	[0.1, 0.3]	0.17	1.185	24.9%
like ★	**[0.2, 0.4]**	**0.24**	**1.271**	**26.7%**
deep	[0.4, 0.1]	0.13	1.139	23.9%
learning	[0.3, 0.2]	0.16	1.174	24.6%

★ = the true context word for this pair. Sanity check: 24.9% + 26.7% + 23.9% + 24.6% = 100%. The model currently gives “like” only 26.7% confidence — training should push this toward 100%.

STEP 3 · LOSS

Cross-Entropy Loss for This Training Pair

$$L = -\log P(\text{like})$$

$$= -\log(0.2666)$$

$$\approx 1.322$$

This is the loss for just ONE pair; the (deep, learning) pair gets its own loss too.

This is the loss for this ONE pair only — the (deep, learning) pair is a separate training example with its own forward pass and its own loss.

STEP 3 · LOSS

Every word also gets an error signal — predicted probability minus the ideal one-hot label:

$$e^w = P(w) - y(w)$$

Word	P(w) − y(w)	e^w	Direction
I	0.249 − 0 =	**+0.249**	push away
like ★	0.267 − 1 =	**−0.733**	pull closer
deep	0.239 − 0 =	**+0.239**	push away
learning	0.246 − 0 =	**+0.246**	push away

Negative error (like) means "pull this output vector closer to v_deep"; positive error (everyone else) means "push it away."

Step 4 · Backward Pass — Update the Weights

The gradient with respect to the target vector sums every word's error, weighted by its output vector:

$$\partial L/\partial v_{deep} = \Sigma_i\, e_i \cdot u_i$$

- $e_i \cdot u_i = 0.249 \times [0.1, 0.3] = [0.0249, 0.0747]$
- $e_{like} \cdot u_{like} = -0.733 \times [0.2, 0.4] = [-0.1466, -0.2932]$
- $e_{deep} \cdot u_{deep} = 0.239 \times [0.4, 0.1] = [0.0956, 0.0239]$
- $e_{learning} \cdot u_{learning} = 0.246 \times [0.3, 0.2] = [0.0738, 0.0492]$

Summing:

x = 0.0249 − 0.1466 + 0.0956 + 0.0738 = 0.0477 ≈ 0.048 ·

y = 0.0747 − 0.2932 + 0.0239 + 0.0492 = −0.1454 ≈ −0.146

$$\partial L/\partial v_{deep} \approx [0.048, -0.146]$$

Update the Target Vector & Output Vectors

Unlike CBOW, this gradient goes to v_deep alone — there's no averaging split across multiple context words, because there was only one input word.

$$\mathbf{v_new = v_old - \eta \times grad}$$

v_deep: $[0.2, 0.5] - 0.1 \times [0.048, -0.146] = \mathbf{[0.195, 0.515]}$

The output vector u_like updates the same way, using its own error directly:

$$\mathbf{u_like = u_like - \eta \cdot e_like \cdot v_deep}$$

u_like: $[0.2, 0.4] - 0.1 \times (-0.733) \times [0.2, 0.5] = \mathbf{[0.215, 0.437]}$

Next, the (deep, learning) pair repeats this exact process independently, producing its own gradient for v_deep. Both pairs' gradients accumulate before v_deep's next update.

One Skip-gram Training Pair, End to End

1

Look Up

Take the target word's own vector from W directly — no averaging needed.

2

Score & Softmax

Compare that vector against every word's output vector; softmax gives a probability per context word.

3

Loss

Cross-entropy loss compares predicted P(context word) against the ideal value of 1, per pair.

4

Update

The error signal updates v_target and the relevant output vectors; each context word contributes its own pair.

Each occurrence of a word generates multiple training pairs — more gradient updates per rare word than CBOW gives it.

CBOW vs. Skip-gram: Real-World Scenarios

The Core Trade-off (CBOW and SKIP-Gram)

CBOW

Context → Target

- Averages context words into one prediction
- Faster to train (fewer examples per window)
- Excels on frequent words / huge corpora
- Best when speed and common-word quality matter most

Skip-gram

Target → Context

- Generates one training pair per context word
- Slower to train, but more signal per occurrence
- Excels on rare / domain-specific words
- Best when data is sparse or the target is niche

CBOW vs Skip-gram

Two training strategies within Word2Vec.

Feature	CBOW	Skip-gram
Input	Context words	Target word
Output	Target word	Context words
Direction	Context → Target	Target → Context
Training Speed	Generally faster	Generally slower
Best For	Frequent words	Rare words

CBOW
Context → Word

Skip-gram
Word → Context

APPLICATION QUESTION 1

E-Commerce: Frequent vs. Rare Products

An online marketplace embeds products from co-purchase sequences (e.g. “laptop → mouse → keyboard”) using millions of transactions. Most products are purchased often (laptop, mouse), but a long tail of niche accessories (e.g. a specific stylus refill) are purchased only a handful of times.

Q: **Which architecture should power embeddings for the frequent catalog, and which for the rare long-tail items?**

ANSWER

Both

one model per segment

WHY

- **Frequent items:** CBOW works well because these items appear many times. It trains faster and still learns good vectors.
- **Rare items**: Skip-gram works better because it creates multiple training pairs for each occurrence, giving rare items more learning opportunities.
- **In practice:** Companies can use CBOW for the full catalog and then use Skip-gram for rare items to improve their representations.

APPLICATION QUESTION 2

Voice Assistant: Tight Compute, Common Phrases

A voice-assistant team trains phrase embeddings nightly on billions of short utterances (“set a timer”, “play music”) inside a strict compute window. The vocabulary is dominated by a small set of very common commands; rare phrasings are not the priority for this release.

Q: Given the volume and the compute ceiling, which architecture fits best — and why not the alternative?

ANSWER

CBOW

WHY

- **CBOW is faster**: It creates only one training example for each context window, so it needs less computation.
- **Common words have enough data:** Since these words appear very often, CBOW can learn good-quality word vectors from the large amount of available data.
- **Skip-gram is not necessary:** Skip-gram creates multiple training pairs, which increases the work. For common words, this extra work usually gives little additional benefit.

APPLICATION QUESTION 3

Legal Tech: Obscure Precedents & Citations

A legal research platform builds embeddings from case law to power “find similar precedent” search. Landmark cases are cited constantly, but many obscure precedents relevant to niche disputes appear only a few times across the entire corpus — yet finding them correctly is the product's core value proposition.

Q: Which architecture should the team prioritize for the embedding model, given what the product needs to get right?

ANSWER

Skip-gram

WHY

- **Rare words are important:** The product needs to find rare and less-known information, so Skip-gram is a better choice.
- **Better learning from few examples:** Skip-gram creates multiple training pairs from each rare word, helping it learn more from limited data.
- **Extra training is worth it**: Skip-gram takes more time than CBOW, but the better search quality for rare information makes it worthwhile.

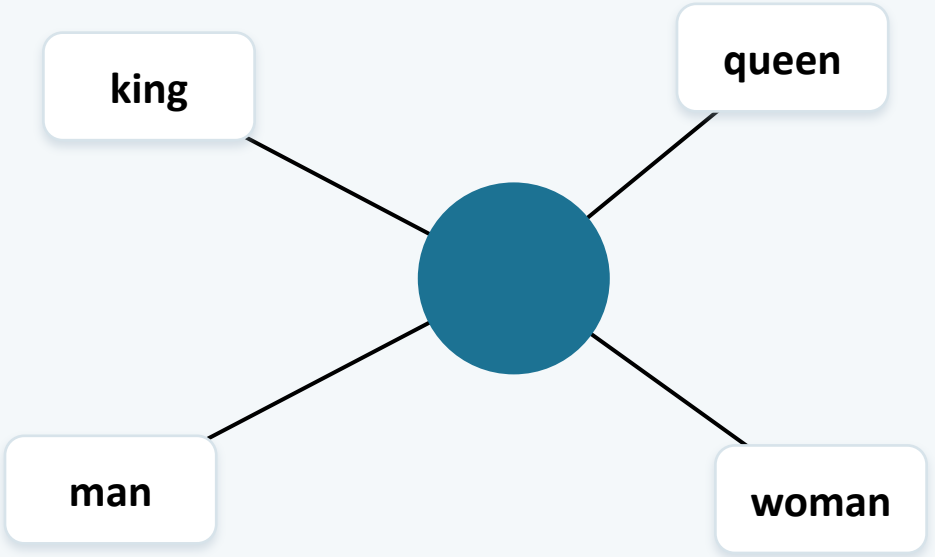
GENSIM

What is Gensim?

Gensim is a Python library used in Natural Language Processing (NLP) to work with a large amount of text.

In means

It helps a computer understand relationships between words and documents.



Words become numbers (vectors) — related words end up close together

Main Uses of Gensim

Word Embeddings

Converts words into numbers (vectors) that capture meaning.

Topic Modeling

Finds the main topics hidden inside a collection of documents.

Document Similarity

Identifies documents that discuss similar things.

Works with Large Text

Built to handle huge text collections efficiently.

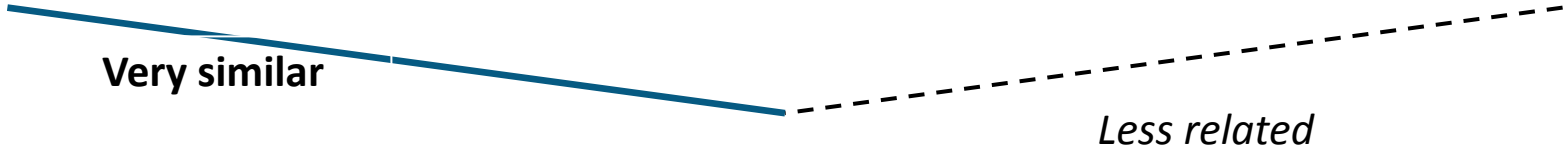
A Simple Example

Imagine you have these three sentences:

“I love playing cricket.”

“Cricket is my favorite sport.”

“I enjoy football.”



Gensim helps the computer understand:

"Cricket" and "sport" are related in meaning.

The first two sentences are more similar to each other.

Football is a sport too, but less related to the first sentence.

Popular Techniques in Gensim

Word2Vec	Learns relationships between words.
FastText	Learns word representations, including parts of words.
Doc2Vec	Creates representations for entire documents.
LDA	Finds hidden topics in documents.
LSI	Identifies relationships between words and documents.

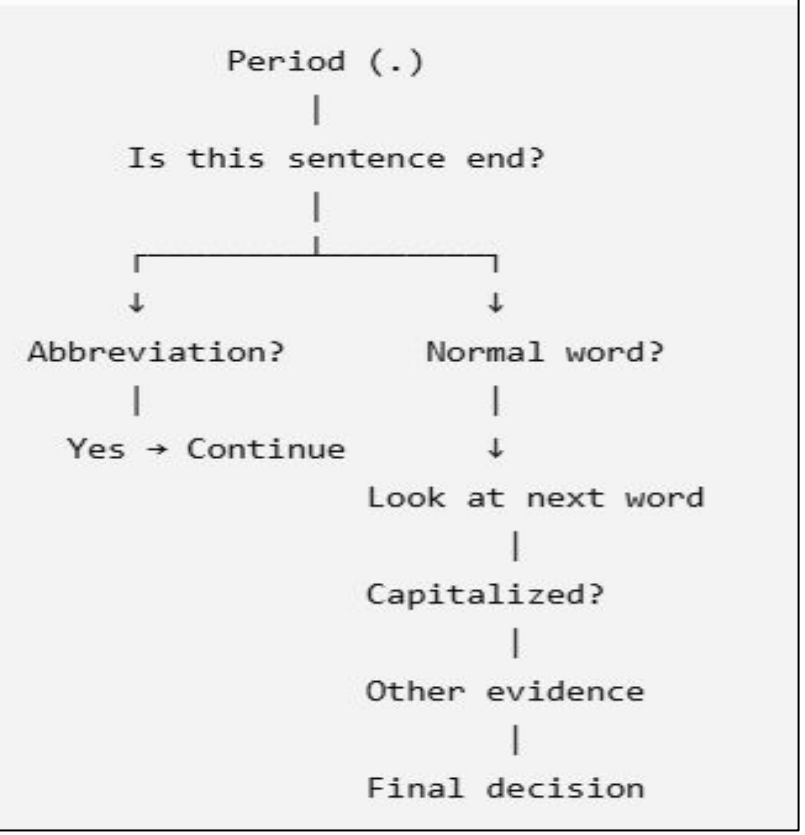
Gensim (Word2Vec) Implementation

NLTK data packages

```
nltk.download('punkt_tab')
nltk.download('stopwords')
```

You'll also need to download a couple of **NLTK data packages** the first time you run this:

Think of PUNKT like this:



to take a paragraph and divide it into individual sentences.

Punkt → a tokenizer algorithm developed for splitting tex

punkt_tab → newer NLTK data files containing the tables/rules needed by the Punkt tokenizer.

It is not a Python library; it is downloaded NLTK data.

NLTK data packages

```
import nltk #This imports the NLTK (Natural Language Toolkit) library.

nltk.download('punkt_tab') #This downloads the PUNKT tokenizer data/model required by NLTK.

from nltk.tokenize.punkt import PunktSentenceTokenizer, PunktParameters
'''importing two things from NLTK PunktSentenceTokenizer This is the actual sentence tokenizer.
and try to divide it into:

Sentence 1
Sentence 2
PunktParameters

This allows us to customize the tokenizer.

For example, we can tell it:

"Miss. and Mr. are abbreviations, so don't treat their periods as sentence endings."'''


text = "We met Miss. Tanaya Das and Mr. Rohan Singh today. They are pursuing a B.Tech degree in Data Science."
```

```
# Tell the tokenizer which abbreviations should NOT trigger a sentence break
punkt_param = PunktParameters()
'''
object called:

punkt_param of type:PunktParameters Think of it as creating a settings box for PUNKT.

Initially:
punkt_param
      ↓
PUNKT settings

We are going to put our abbreviation information into this settings box.'''

punkt_param.abbrev_types = set(['mr', 'mrs', 'ms', 'miss', 'dr', 'prof', 'sr', 'jr'])

'''telling PUNKT:

These words are abbreviations. Don't automatically split the sentence after their periods.'''
```

NLTK data packages

```
tokenizer = PunktSentenceTokenizer(punkt_param)
'''
Now we create the actual PUNKT tokenizer.

We give it our customized settings:

punkt_param
     ↓
PunktSentenceTokenizer
     ↓
Customized tokenizer

So the tokenizer now knows:

mr     → abbreviation
mrs    → abbreviation
miss   → abbreviation
dr     → abbreviation
prof   → abbreviation'''


sentences = tokenizer.tokenize(text) #divide this text into sentences
print(sentences)
```

```
['We met Miss. Tanaya Das and Mr. Rohan Singh today.', 'They are pursuing a B.Tech degree in Data Science.']
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Setup — Install & Import

NLTK: It provides tools and datasets to help computers parse, process, and understand human language for tasks like sentiment analysis, text classification, and data cleaning

```
# Word2Vec with Gensim – Space Exploration Corpus
# ============================================================



import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from gensim.models import Word2Vec
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

nltk.download('punkt_tab')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

What each library is for

gensim

Trains the Word2Vec model on our text.

nltk (natural language toolkit)

Tokenizes sentences and supplies English stopwords.

scikit-learn (PCA)

Reduces 100-dim vectors to 2D for plotting.

matplotlib

Draws the final scatter plot of word vectors.

Step 1 — Build the Corpus

```
# ------------------------------------------------------------
# Step 1: A beautiful little corpus (12 sentences, one theme)
# ------------------------------------------------------------
corpus = [
    "The rocket launched into orbit carrying three astronauts.",
    "Astronauts train for years before their first mission to space.",
    "The International Space Station orbits the Earth every ninety minutes.",
    "Mars has long fascinated scientists searching for signs of ancient water.",
    "NASA and SpaceX are working together to send humans back to the moon.",
    "The moon landing in 1969 remains one of humanity's greatest achievements.",
    "Telescopes like Hubble capture stunning images of distant galaxies.",
    "A galaxy contains billions of stars bound together by gravity.",
    "Gravity keeps planets locked in orbit around their host star.",
    "Scientists study asteroids to understand the early solar system.",
    "The solar system includes eight planets orbiting a single sun.",
    "Engineers design spacecraft to withstand extreme heat during reentry.",
]
```

Step 2 — Preprocess the Text

```
def preprocess_text(text):
  stop_words = set(stopwords.words('english'))
  tokens = word_tokenize(text.lower())
  tokens = [w for w in tokens
            if w.isalpha()
            and w not in stop_words]
  return tokens

preprocessed_corpus = [preprocess_text(s)
                       for s in corpus]
```

OUTPUT

```
Sample preprocessed sentence:
['rocket', 'launched', 'orbit', 'carrying',
 'three', 'astronauts']
```

THREE CLEANING RULES

Lowercase everything

“Rocket” and “rocket” become the same token.

Keep only alphabetic tokens

isalpha() drops numbers and punctuation.

Remove stopwords

Common filler words (“the”, “and”, “a”) are dropped so they don’t drown out meaningful words.

Step 3 — Train the Word2Vec Model

```
model = Word2Vec(
  sentences=preprocessed_corpus,
  vector_size=100, # length of each vector
  window=5,          # neighboring words
  min_count=1,       # keep rare words too
  workers=4,         # parallel threads
  sg=1               # 1=Skip-gram, 0=CBOW
)
model.save("word2vec_space.model")
```

OUTPUT

```
Vocabulary size: 78 words
Vocabulary: ['system', 'solar', 'planets',
 'gravity', 'moon', 'scientists', ... ]
```

KEY PARAMETERS

`vector_size = 100`

Each word becomes a 100-number vector.

`window = 5`

Looks at up to 5 words on each side for context.

`min_count = 1`

Corpus is tiny, so even 1-occurrence words are kept.

`sg = 1 (Skip-gram)`

Predicts surrounding words from a given word — better for small datasets.

Step 4 — Evaluate the Model

```
sim = model.wv.similarity('galaxy', 'star')

similar = model.wv.most_similar(
  'space', topn=5
)

analogy = model.wv.most_similar(
  positive=['moon', 'orbit'],
  negative=['earth'],
  topn=1
)
```

OUTPUT

```
similarity('galaxy', 'star')
→ 0.0781

most_similar('space', topn=5)
orbiting  0.319
international  0.239
distant  0.204
searching  0.199
ninety  0.196

moon + orbit - earth
→ keeps  (0.205)
```

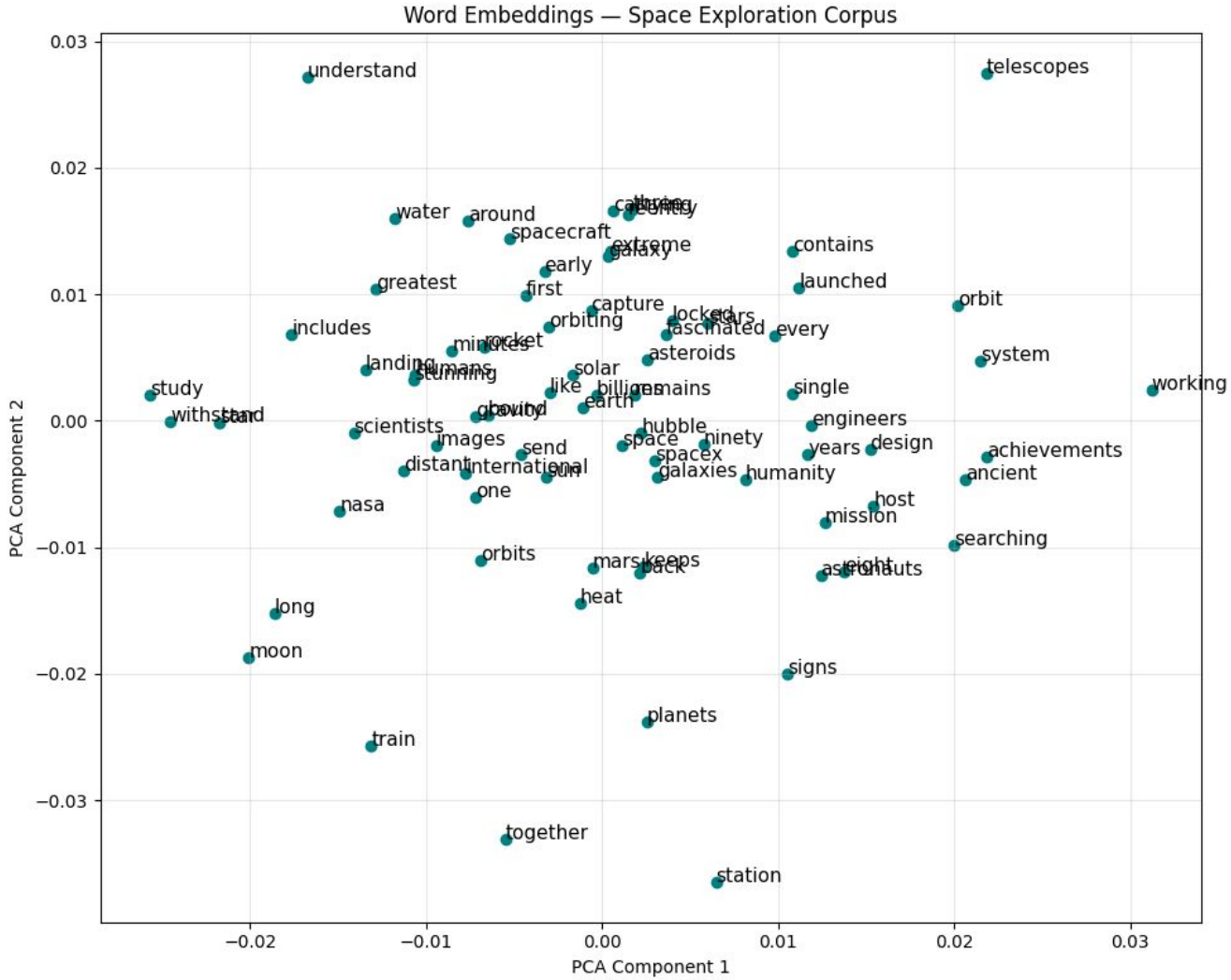
Step 5 — Visualize with PCA

```
# ------------------------------------------------------------
# Step 5: Visualize the embeddings with PCA
# ------------------------------------------------------------
word_vectors = model.wv[model.wv.index_to_key]
pca = PCA(n_components=2)
result = pca.fit_transform(word_vectors)

plt.figure(figsize=(10, 8))
plt.scatter(result[:, 0], result[:, 1], color='teal')

words = list(model.wv.index_to_key)
for i, word in enumerate(words):
    plt.annotate(word, xy=(result[i, 0], result[i, 1]), fontsize=11)

plt.title("Word Embeddings — Space Exploration Corpus")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



PCA compresses each word's 100-number vector down to just 2 numbers (x, y) so it can be plotted — words used in similar contexts land near each other.

The Full Pipeline

From raw sentences to a visual map of word meaning

Corpus	Preprocess	Train	Evaluate	Visualize
12 space-themed sentences	Lowercase, tokenize, remove stopwords	Word2Vec (Skip-gram, 100-dim vectors)	Similarity, most-similar, analogies	PCA → 2D scatter plot

Result: 78 words, each a 100-dimensional vector, positioned so that words appearing in similar contexts sit closer together.

Beyond Word2Vec

Other approaches to building dense word vectors

GloVe — Global Vectors

Learns vectors from global word co-occurrence statistics across the entire corpus, blending the strengths of count-based and prediction-based methods.

FastText

Represents each word as a bag of character n-grams, so it can build embeddings for misspelled or unseen (OOV) words by composing sub-word pieces.

Contextual Embeddings — ELMo / BERT

Generate a different vector for the same word depending on its sentence context (e.g., river “bank” vs. a “bank” account) — covered in a later unit.

GloVe

Word Vectors 2 — and the Problem of Word Senses

What Makes GloVe Different?

Both learn word vectors — but from very different signals in the text.

Word2Vec

Predictive · Local context

Slides a small window over the text and predicts a word from its neighbors (or vice versa). It only ever sees a few words at a time.

"the quick ___ jumps" → "fox"

Never sees the whole corpus at once

GloVe

Count-based · Global statistics

First counts how often every pair of words appears together across the ENTIRE corpus, then learns vectors that match those global counts.

count("fox", "quick") across all text → vector

Uses the whole corpus's statistics at once

What Is GloVe?

GloVe (Global Vectors for Word Representation) is an **unsupervised learning algorithm** that turns words into dense numeric vectors by analyzing how often words co-occur across a large text corpus — capturing the semantic relationships between them.

Scenario for GloVe

Imagine we have these sentences:

"I like apple."
"I like mango."
"I eat apple."
"I eat mango."

GloVe checks which words appear together and how often.

For example:

- apple → appears with *like, eat*
- mango → appears with *like, eat*

Because apple and mango occur in similar contexts, GloVe learns that they are semantically similar.

Pre-Trained GloVe Data

Stanford NLP provides ready-to-use vectors, pre-trained on massive corpora — no training required to get started.

6B

Tokens

Trained on 6 billion words from Wikipedia + Gigaword

400K

Vocabulary

Unique word types covered in the vocabulary

4

Dimensions

Available as 50d, 100d, 200d, or 300d vectors

1

Download

A single flat text file — one word + vector per line

Choosing a Dimension

Dimension	File	Typical Use
50d	glove.6B.50d.txt	Fast prototyping, small models, classroom demos
100d	glove.6B.100d.txt	Balanced accuracy and speed for coursework
200d	glove.6B.200d.txt	Richer semantics for mid-size classification tasks
300d	glove.6B.300d.txt	Best accuracy; heavier memory and compute cost

Directly Downloadable

Vectors can be downloaded and loaded directly into an NLP pipeline — no need to train from scratch for common tasks.

Pre-Trained GloVe Data

Example of Pre-trained GloVe

Suppose you want to build a sentiment analysis model.

Your sentence is:

“**The movie was excellent.**”

Instead of creating word vectors yourself, you download the pre-trained

GloVe 50d file.

Pre-Trained GloVe Data

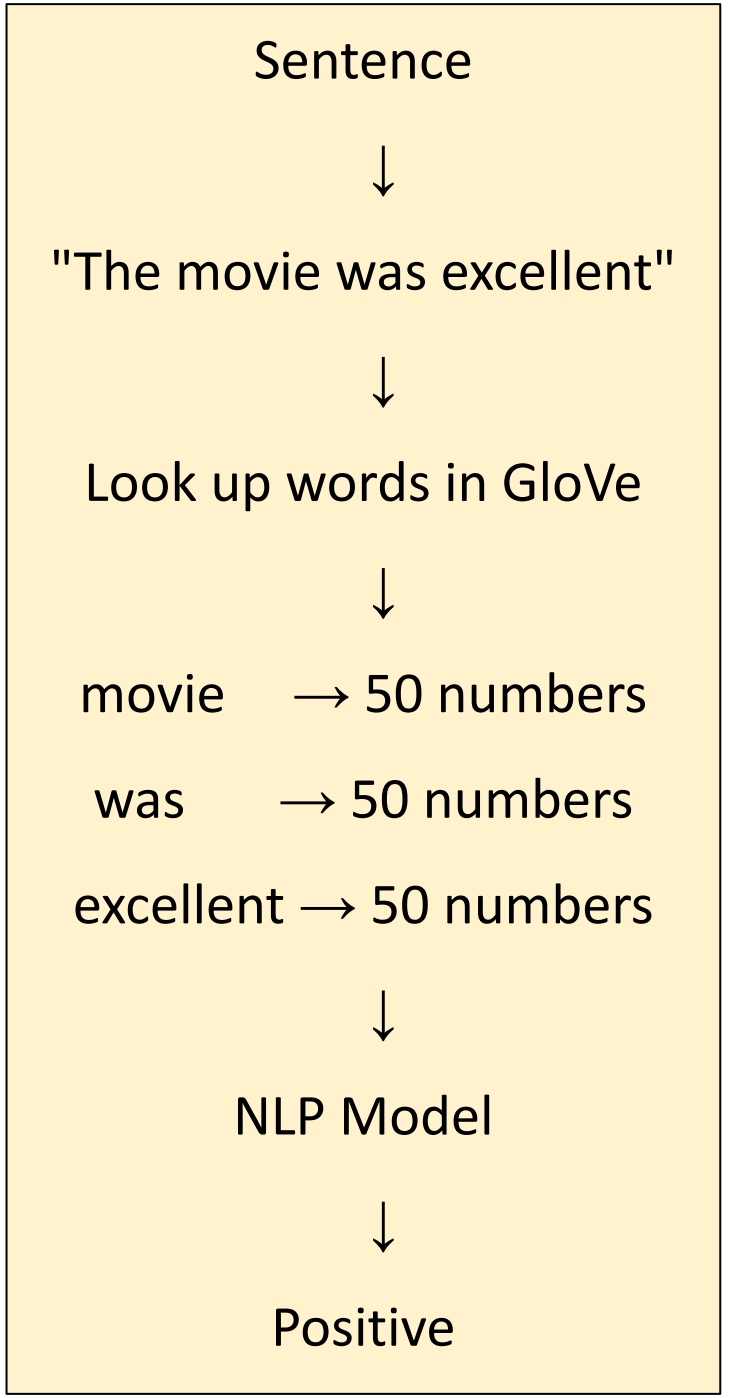
GloVe already has vectors for words:

movie → [0.21, 0.35, 0.17, ... 50 numbers]

excellent → [0.62, 0.48, 0.71, ... 50 numbers]

Your model uses these vectors as the input representation

of the words.



Pre-Trained GloVe Data

Why is it useful?

GloVe has already learned relationships between words from billions of words.

For example:

king ↔ queen

man ↔ woman

cat ↔ dog

Words with similar usage tend to have similar vectors.

So, pre-trained GloVe = download ready-made word vectors and directly use them in NLP model.

The GloVe Pipeline

Six stages turn raw text into a trained embedding matrix. The next slides walk through each one with a worked example.

1. Preprocess	2. Vocabulary	3. Co-occurrence	4. Dot Product	5. Training	6. Embeddings
Tokenize raw text into words	Collect unique words + frequency	Build the co-occurrence matrix	Compare vectors via dot product	Optimize vectors against PMI	Output the final dense vectors

Example

"Students train the model using data"

Steps 1–2: Preprocess & Build Vocabulary

Tokenization splits raw text into individual words. Then we build a **vocabulary** — every unique word, counted by how often it appears.

Input Sentence

```
"Students train the model using data"
```

6 tokens after splitting on whitespace

Tokenized List

```
['Students', 'train', 'the',
'model', 'using', 'data']
```

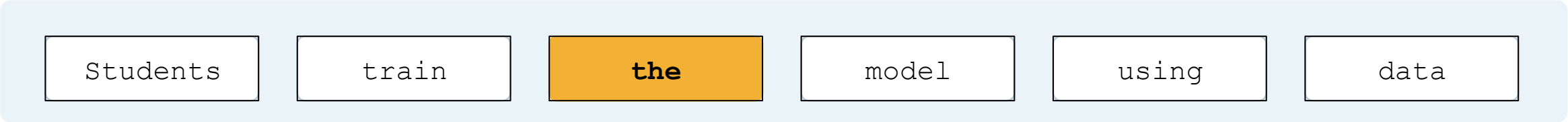
Vocabulary with Frequencies

Word	Index	Frequency
Students	1	1
train	2	1
the	3	1
model	4	1
using	5	1
data	6	1

Every word here is unique, so vocabulary size = 6 and each frequency = 1.

Step 3: Building the Co-Occurrence Matrix

Using a window of 2 words on each side, we count how often every pair of words appears near each other.



window = 2 → for "the", context = {Students, train, model, using}

	Students	train	the	model	using	data
Students	0	**1**	**1**	0	0	0
train	**1**	0	**1**	**1**	0	0
the	**1**	**1**	0	**1**	**1**	0
model	0	**1**	**1**	0	**1**	**1**
using	0	0	**1**	**1**	0	**1**
data	0	0	0	**1**	**1**	0

Reading the Matrix

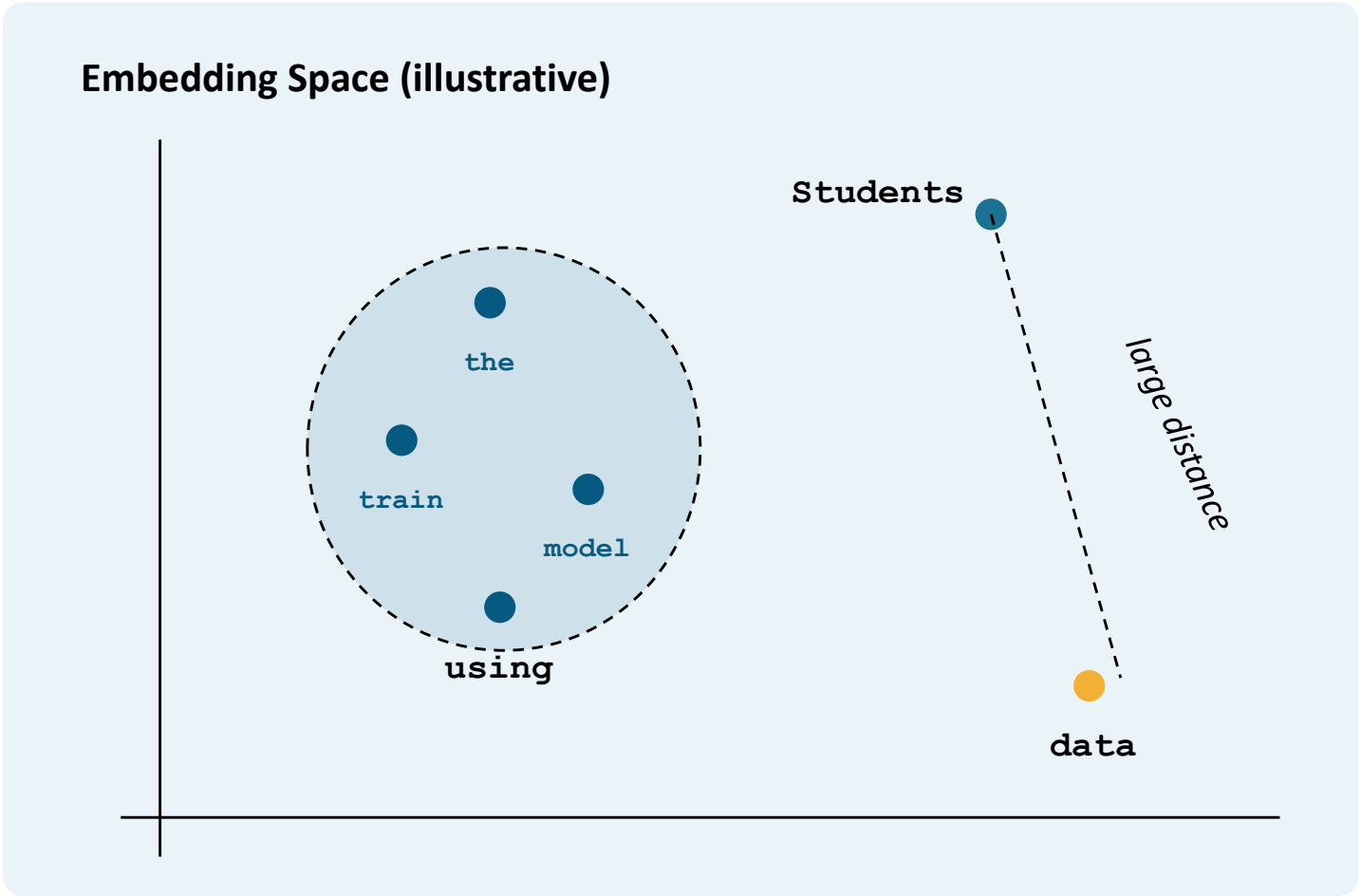
Cell (i, j) = how often word i and word j appear together in the window.

"the" co-occurs with 4 other words — it sits at the center of this short sentence.

"Students" and "data" never co-occur, so their cell is 0 — they're far apart in the sentence.

Step 4: The Dot Product Intuition

The goal: learn vectors so that **dot product ≈ co-occurrence strength.** Words that frequently appear together end up with similar vectors — words that rarely do end up far apart.



What the Vectors Learn

`"the" & "train"`

Co-occur often → Vectors trained close together

`"the" & "model"`

Co-occur often → Vectors trained close together

`"Students" & "data"`

Never co-occur → Vectors trained far apart

Step 5: Training the Word Vectors

GloVe adjusts every word vector so its dot product matches the observed co-occurrence statistics as closely as possible.

Back to Our Sentence

"the" co-occurs with 4 words, so training pulls it toward the center of the embedding space — close to train, model, and using.

"Students" and **"data"** share zero co-occurrence in this sentence, so training pushes their vectors apart — though a full corpus of millions of sentences would place them more accurately.

This is why GloVe needs a huge corpus (6B+ tokens) — one sentence gives weak signal; billions give reliable structure.

Step 6: The Final Embedding Matrix

The values are **learned automatically during training**.

```
Random values
      ↓
Model predicts context words
      ↓
Calculates error
      ↓
Gradient descent updates vectors
      ↓
Repeats many times
      ↓
Final vector
```

For example:

Word	Vector (3D, illustrative)
Students	[0.12, 0.44, -0.08]
train	[0.31, 0.27, 0.19]
the	[0.29, 0.31, 0.22]
model	[0.33, 0.25, 0.24]
using	[0.27, 0.29, 0.20]
data	[0.09, -0.12, 0.41]

```
Students → [0.12, 0.44, -0.08]
```

These numbers are the result of **many training updates**. They are **not manually assigned**, and each dimension does not have a specific human meaning.

Step 6: The Final Embedding Matrix

After training, every word in the vocabulary is represented by one dense vector — this collection is the embedding matrix.

Word	Vector (3D, illustrative)
Students	[0.12, 0.44, -0.08]
train	[0.31, 0.27, 0.19]
the	[0.29, 0.31, 0.22]
model	[0.33, 0.25, 0.24]
using	[0.27, 0.29, 0.20]
data	[0.09, -0.12, 0.41]

What This Matrix Gives You

One row per vocabulary word — shape (vocab_size, embedding_dim)

Dense, low-dimensional vectors instead of huge sparse one-hot vectors

Captures semantic AND syntactic relationships between words

Directly pluggable as a Keras/PyTorch embedding layer

Applications of GloVe

Text Classification

Sentiment analysis, topic tagging, spam detection

Named Entity Recognition

Better entity boundaries via word relationships

Machine Translation

Aligns source/target word representations

Question Answering

Understands context for accurate answers

Document Similarity

Semantic clustering and retrieval



Word Analogy

"king − man + woman = queen"-style reasoning

Does Vector Size Matter?

50 dimensions

Small file, fast to load, low memory.

Similarity scores are usable but coarse — related words score high, but the ranking of 'most similar' words can feel a little rough around the edges.

Good for: prototyping, small projects, limited memory.

300 dimensions

Larger file, slower to load, more memory.

The same similarity tests come back noticeably sharper — related words score higher, unrelated words score lower, and rankings feel more reliable.

Comparison of Major Techniques

Technique	Main Idea	Example
TF-IDF	Importance of a word in documents	*"Python" is important in a Python document*
Co-occurrence	Words appearing together	*"language" ↔ "processing"*
CBOW	Predict word from context	*Context → "drinks"*
Skip-gram	Predict context from word	*"drinks" → Context*
FastText	Uses sub-word information	*playing → play, lay, etc.*
GloVe	Uses global co-occurrence statistics	*word relationships from corpus*

Word Window Classification

Word Window Classification is a method that uses a word and its surrounding context to predict the word’s class or tag.

One word, many meanings

A single word embedding is fixed — it cannot tell whether "**Paris**" names a city or a person until it sees the words around it.

- Word2Vec / GloVe assign one vector per word type, regardless of context.
- Tasks like Named Entity Recognition need a label per token occurrence, not per word type.
- The fix: look at the words surrounding the target — its "window" of context.

SENTENCE

Paris Hilton arrived at the gala.

→ **PERSON**

SENTENCE

The museum in **Paris** is closed.

→ **LOCATION**

Same word. Same vector. Two different labels. That's the gap word window classification is built to close.

What is Word Window Classification?

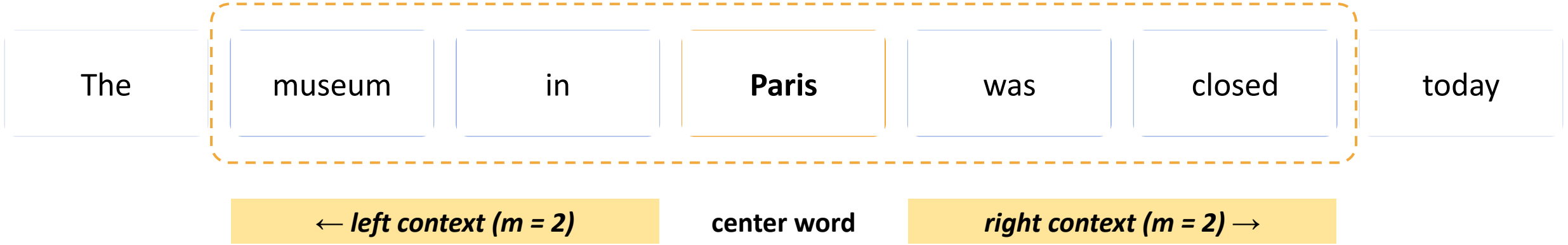
Instead of classifying a word alone, we classify it together with a fixed number of neighbouring words on each side — its "window". The window supplies the context a single vector lacks.

Center word	Window size (m)	Label
The token we want to classify — e.g. decide if it is a LOCATION.	How many words to include on each side. m = 2 means 2 left + 2 right neighbors.	The output class assigned to the center word: PERSON, LOCATION, ORG, OTHER …

Formally: given a sentence $w_1, w_2, \ldots, w_{\square}$ and window size m, classify each w_i using the sub-sequence $[\, w_{i-\square}, \ldots, w_i, \ldots, w_{i+\square} \,]$

A window of size m = 2 around “Paris”

The window always has an odd total length: m words to the left, the center word, and m words to the right.



Edge case: when the center word is near the start or end of a sentence, the window is padded with a special <START>/<END> token vector so every window has the same fixed length.

The center word is decided by sliding the window across the sentence.

For m = 2, we take:

- 2 words before
- 1 center word
- 2 words after

Example:

The museum in Paris was closed today

The museum in Paris was closed today

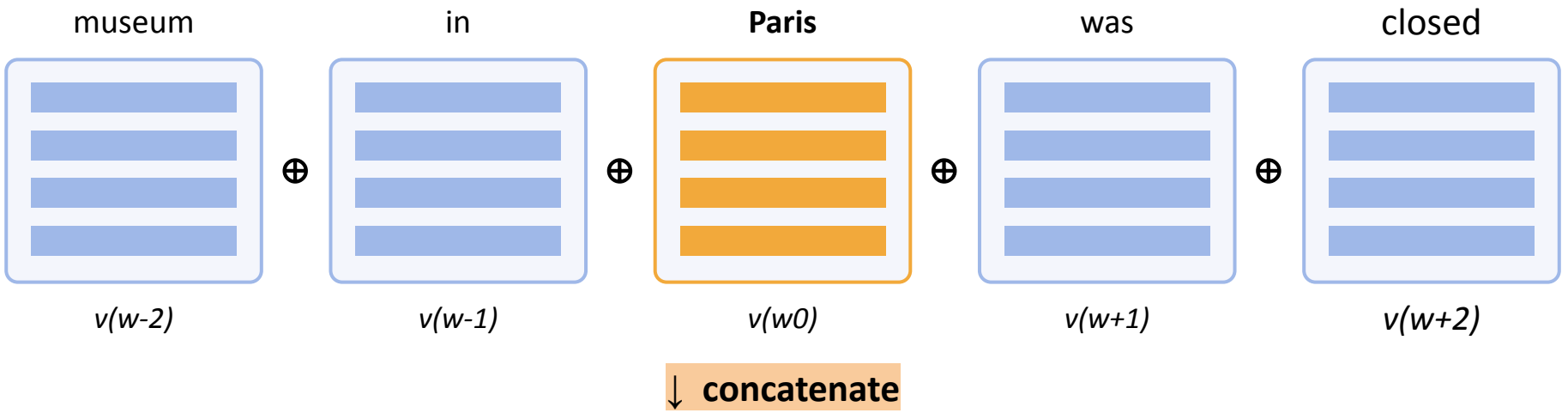
Window	Center word
`<START> <START> The museum in`	The
`<START> The museum in Paris`	museum
`The museum in Paris was`	in
`museum in Paris was closed`	Paris
`in Paris was closed today`	was
`Paris was closed today <END>`	closed
`was closed today <END> <END>`	today

So, **every word in the sentence becomes the center word one by one.**

STEP 1 — REPRESENTATION

Concatenating the window into one vector

Each word in the window is looked up in an embedding matrix, then all vectors are concatenated end-to-end into a single input vector x_window.



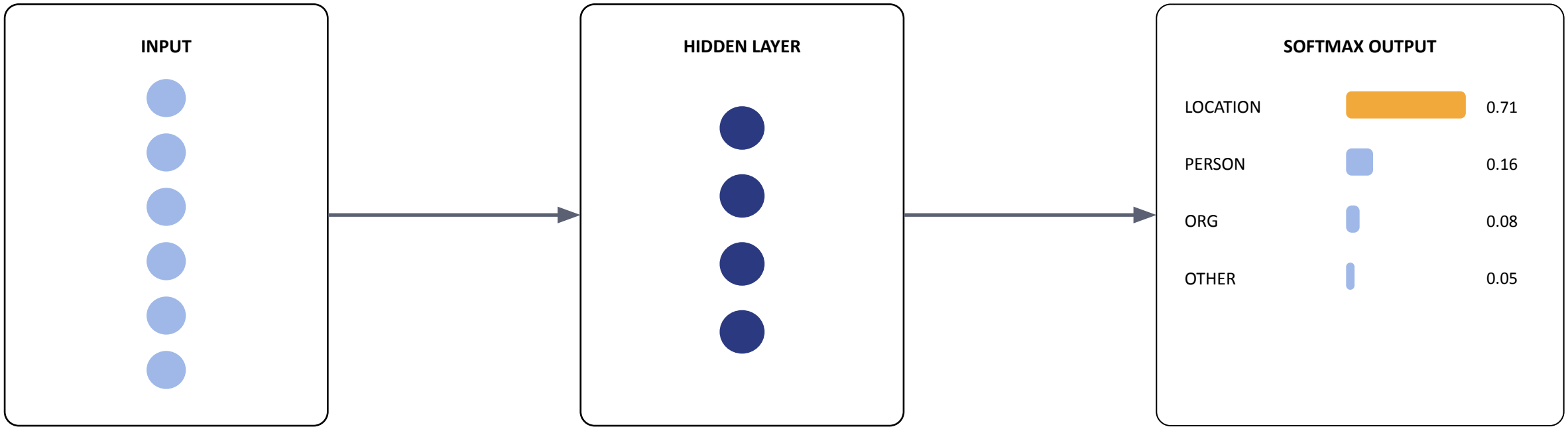
$x_window = [\, v(w_{-2}) ; v(w_{-1}) ; v(w_0) ; v(w_{+1}) ; v(w_{+2}) \,]$ — a single (5 × d)-dimensional vector

If each word vector has dimension d = 100 and the window has 5 words, x_window has 500 dimensions.

STEP 2 — THE CLASSIFIER

A simple feed-forward network

The concatenated window vector passes through a hidden layer, then a softmax layer produces class probabilities.



f = tanh (or ReLU) — a non-linear activation lets the network combine window words in non-trivial ways.

We use tanh or ReLU as an activation function in the hidden layer to help the network learn complex relationships between the words in the window.

ReLU: keeps positive values and changes negative values to 0.
tanh: converts values to a range between −1 and +1.

The main goal is to classify the center token using its surrounding words.

Example:

museum in [Paris] was closed

↑

Center Token

1. Create window → Take surrounding words.
2. Concatenate vectors → Combine their word vectors.
3. Hidden layer → Learns relationships between the words.
4. Softmax → Converts raw scores into probabilities.
5. Highest probability → Final class.

The network looks at the center token and its context, then predicts the most likely class for that token.

End-to-end: classifying “Paris” as LOCATION

Sentence	“The museum in Paris was closed today.” — target token: Paris
Build window (m = 2)	[museum, in, Paris, was, closed]
Look up & concatenate	x_window = [v(museum) ; v(in) ; v(Paris) ; v(was) ; v(closed)] → 500-d vector
Hidden layer	h = tanh(W · x_window + b) — learns which combinations of context words signal a place
Softmax scores	$\hat{y}$ = softmax(U · h) → LOCATION 0.71, PERSON 0.16, ORG 0.08, OTHER 0.05
Prediction	argmax($\hat{y}$) = LOCATION — correct, because “museum” and “closed” are typical place-context words

STEP 3 — LEARNING

Training with cross-entropy loss

The network learns by comparing predicted probabilities ŷ to the true label y, then updating every parameter—including the word vectors themselves.

CROSS-ENTROPY LOSS

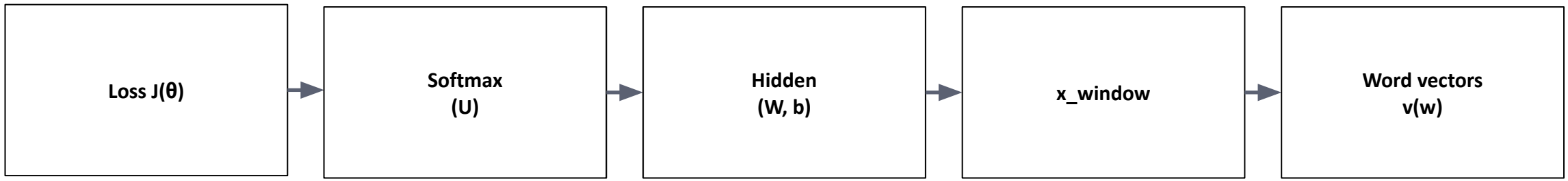
$$J(\theta) = -\sum_i y_i \log(\hat{y}_i)$$

Penalizes the model when the probability mass on the correct class is low.

PARAMETERS UPDATED (θ)

- U, W, b — the classifier's weights
- v(w) — every word vector in the window, including the center word

Gradient flow (backpropagation)



← gradients flow backward from the loss, updating every box along the way, every training step

Why is learning needed?

Suppose the window is:

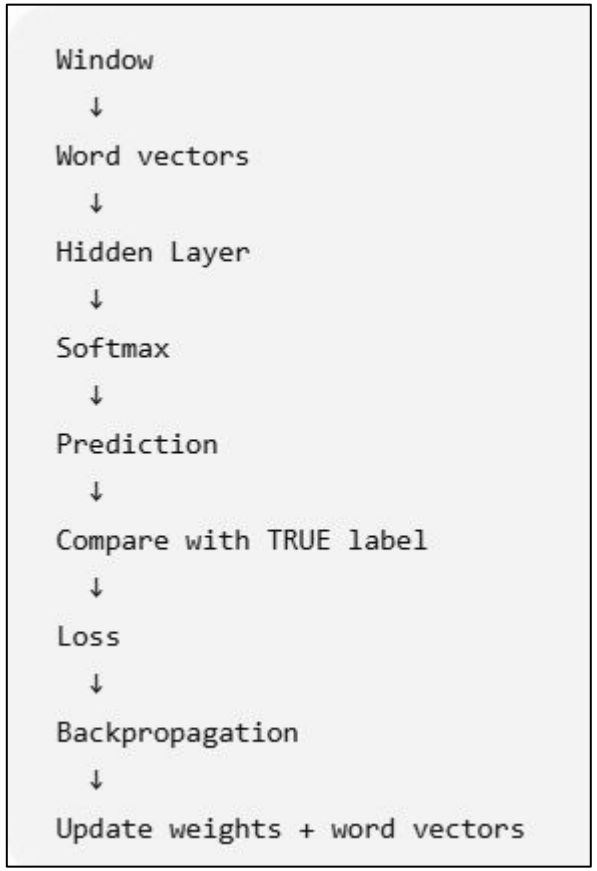
museum in [Paris] was closed
↑
Center token

We want: **Paris → LOCATION**

But initially, the model's weights and word vectors are not trained for this task. So it may predict:

LOCATION → 0.20
PERSON → 0.40 (which is wrong predicted)
ORG → 0.25
OTHER → 0.15

The true answer is LOCATION.



What does training do?

1. The model makes a prediction
2. Cross-entropy loss checks how wrong the prediction is.
3. Backpropagation sends the error backward.
4. Weights and word vectors are updated.
5. The model tries again and gradually becomes better.

Why update the window's word vectors?

Because the meaning of the center word depends partly on its context.

For example:

in [Paris] was (museum in [Paris] was closed)

- helps the model learn that Paris is likely a LOCATION.
- Over many examples, training adjusts the word vectors so that they become more useful for predicting the correct class.

means:

Learning is needed so the model can use the words in the window to gradually learn how to correctly classify the center token.

Applications in NLP

Named Entity Recognition

Label each token as PERSON, LOCATION, ORG, or O (none).

Part-of-Speech Tagging

Classify each word's grammatical role using nearby words.

Chunking

Group tokens into noun/verb phrases using local context windows.

Word Sense Disambiguation

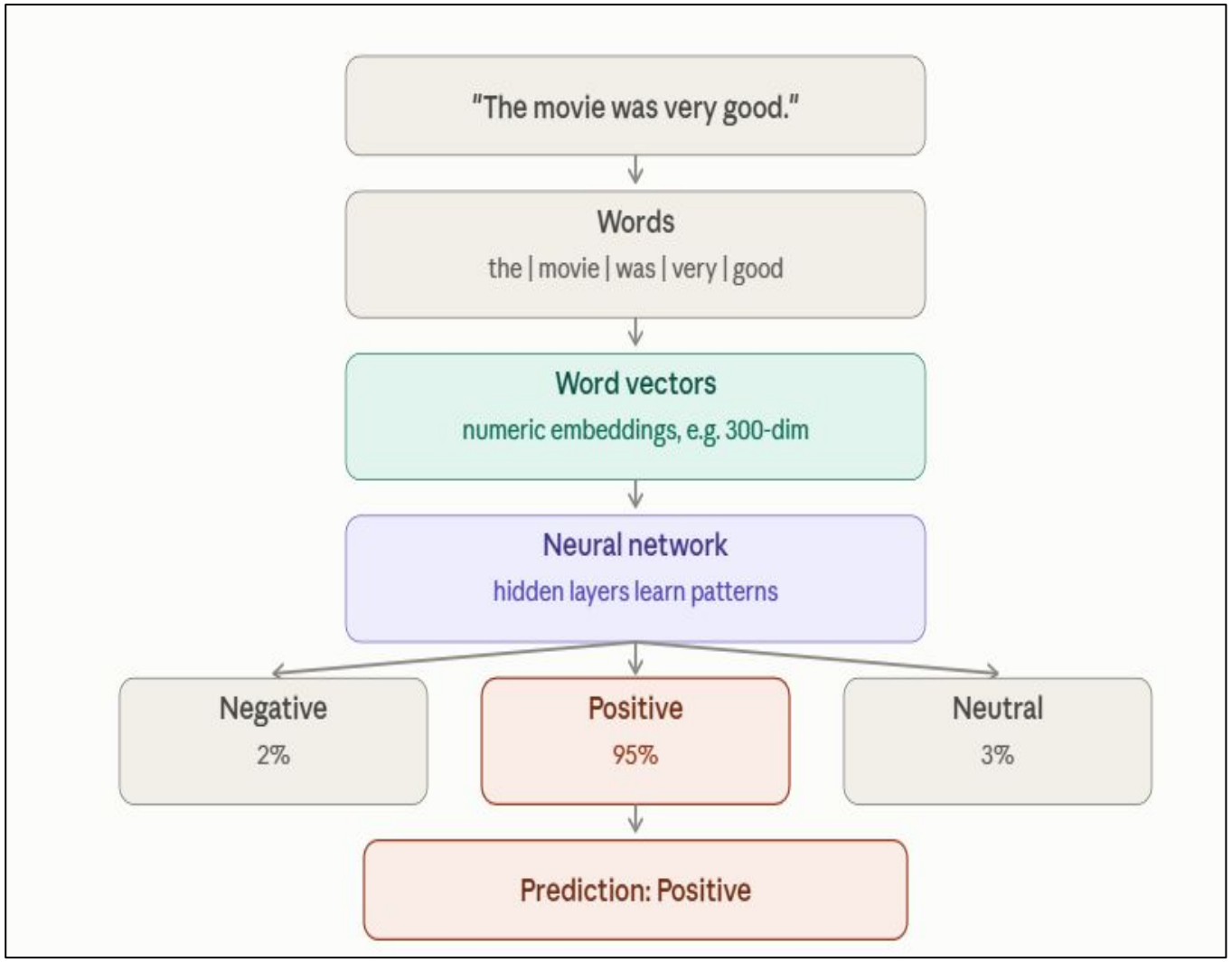
Decide which meaning of a word applies, given its neighbors.

Neural Networks, Matrix Calculus & Backpropagation in NLP

Neural Networks in NLP

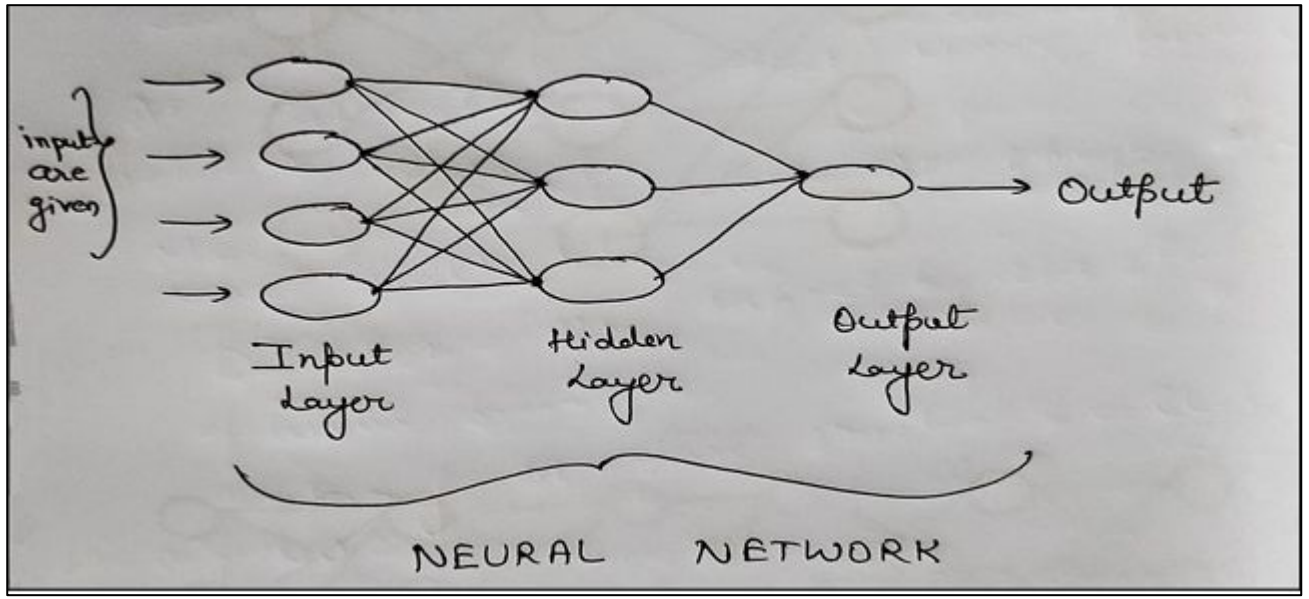
Neural networks learn patterns from word vectors and their context to perform NLP tasks such as classification, sentiment analysis, and NER.

Words → Word Vectors → Neural Network → Prediction



Neural Networks: Structure and Learning

A neural network is made of artificial neurons arranged in layers. It takes input data, learns patterns, and produces an output.



Types of Learning

- Supervised Learning: Learns from labeled data to make predictions.
- Unsupervised Learning: Finds patterns in unlabeled data.
- Reinforcement Learning: Learns through feedback or rewards.

Building Blocks of a Neural Network

1. Neurons

A neuron receives inputs, performs calculations, and produces an output.

Neuron processing:

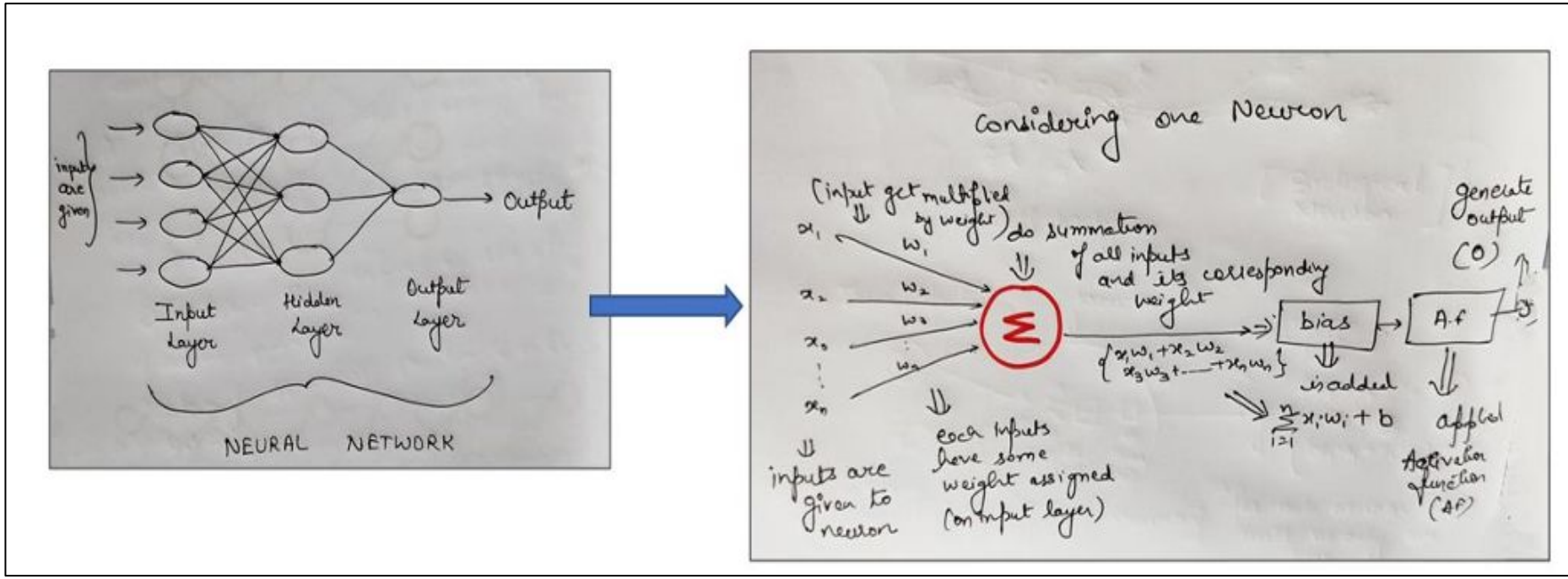
Inputs ⟶ Weighting ⟶ Summation + Bias ⟶ Activation Function ⟶ Output

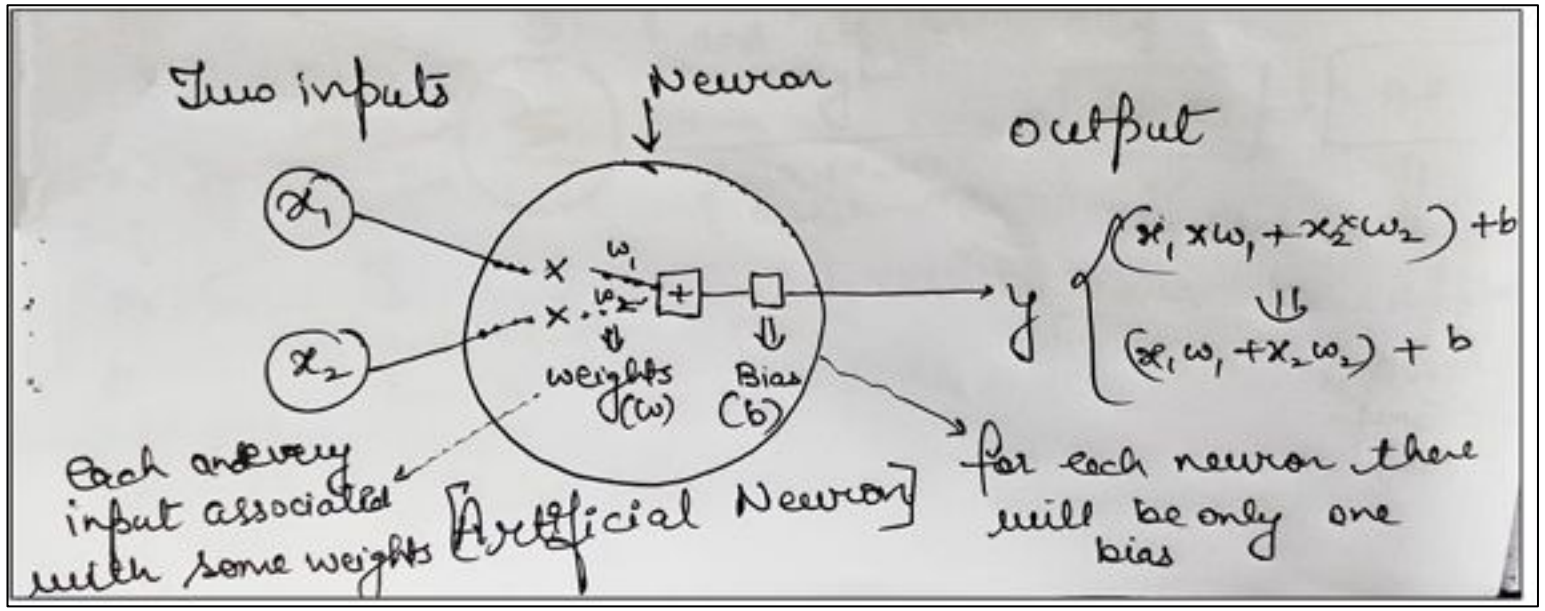
Mathematically:

$$z = w_1x_1 + w_2x_2 + b$$

Output = Activation(z)

- **Weights**: Determine the importance of each input.
- **Bias:** Adjusts the neuron's baseline.
- **Activation Function:** Introduces non-linearity and helps learn complex patterns.





Building Blocks of a Neural Network

2. Layers

Neurons are organized into layers:

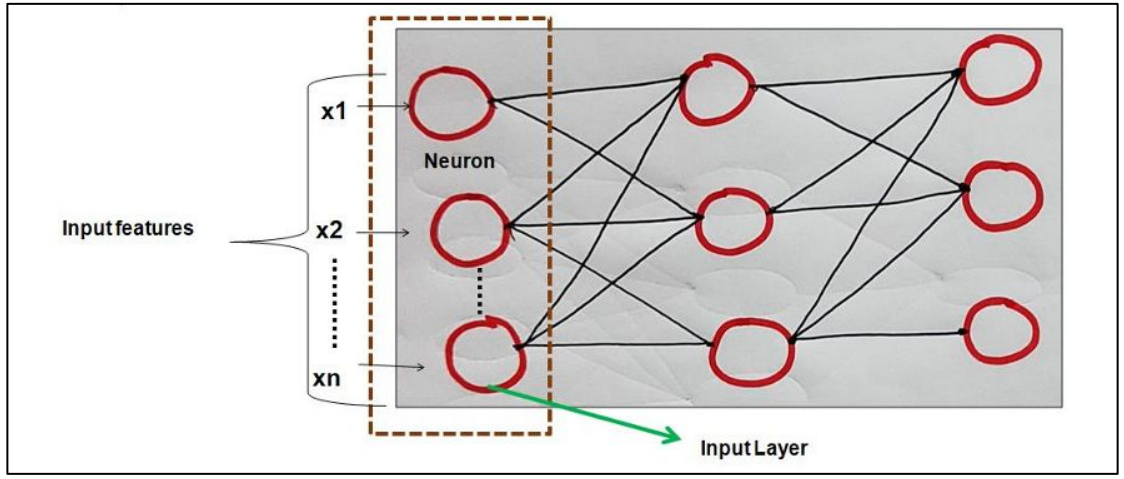
Input Layer → Hidden Layer(s) → Output Layer

Together, neurons and layers form the basic structure of a neural network.

Terminologies Used in Neural Networks

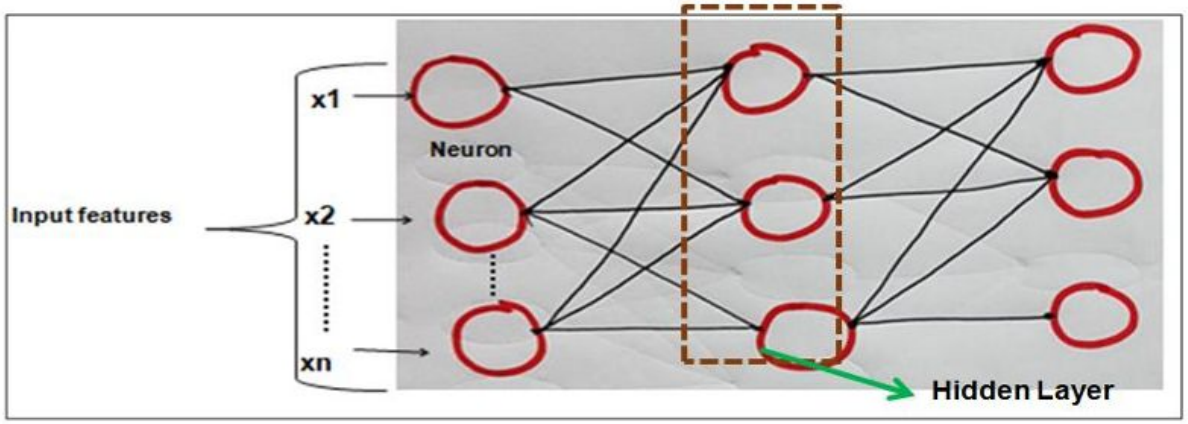
1. Input Layer

- Receives the input features.
- Number of neurons = number of input features.
- Example: 11 features ⟶ 11 input neurons.



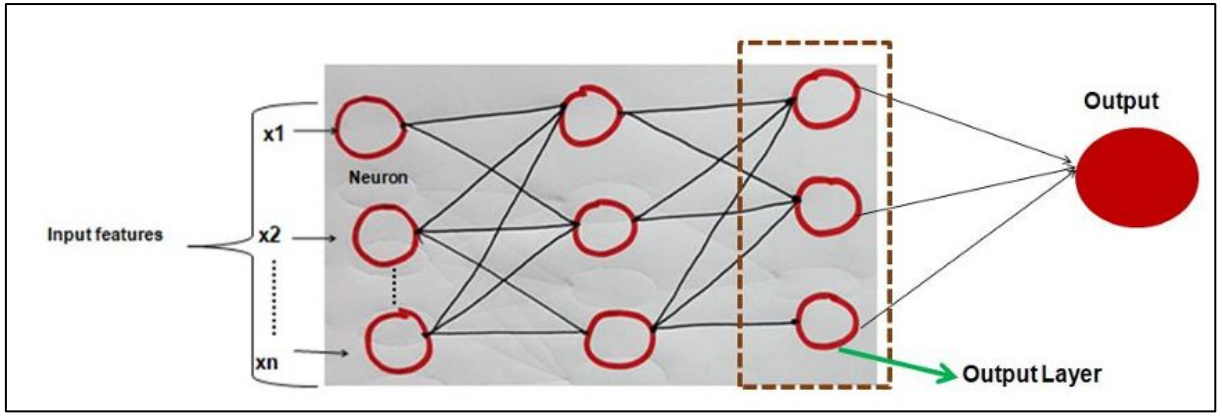
2. Hidden Layer

- Performs learning and pattern detection.
- Can have one or more hidden layers with different numbers of neurons.
- Too many neurons ⟶ Overfitting
- Too few neurons ⟶ Underfitting
- The best structure is usually found through experimentation.



3. Output Layer

- Produces the final prediction.
- Number of neurons depends on the task:
  - Binary classification ⟶ 1 neuron
  - Multi-class classification ⟶ Multiple neurons
- The activation function is selected according to the problem.



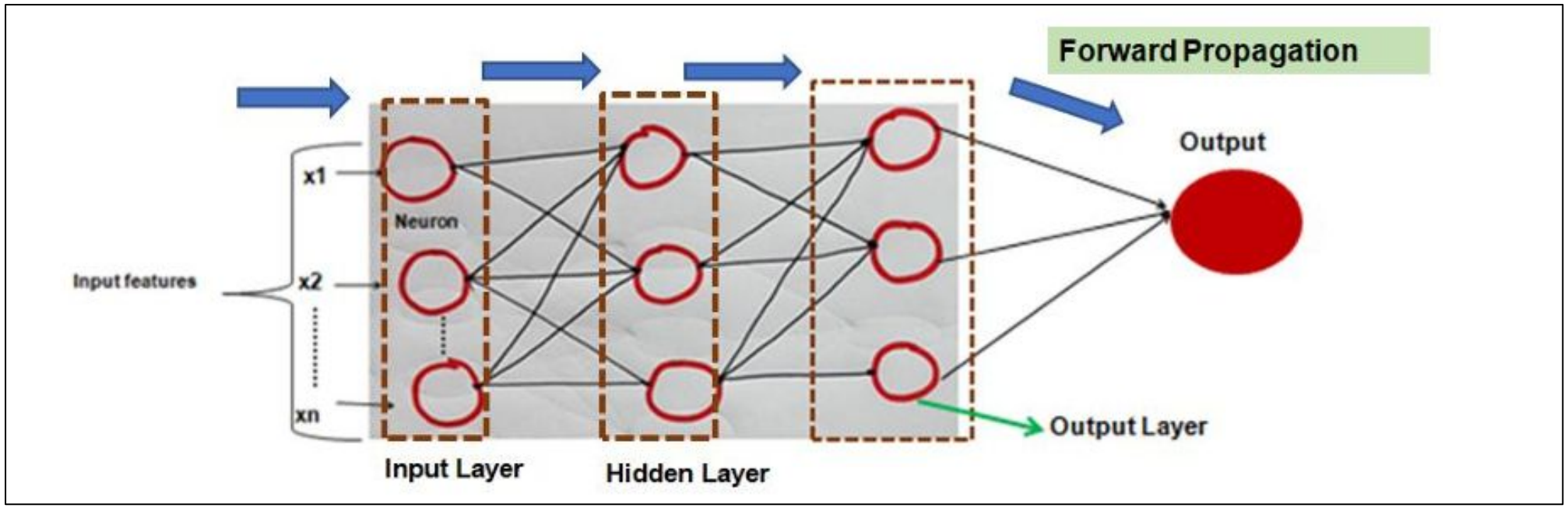
1. Forward Propagation

Forward propagation (forward pass) is the process of moving input data through the neural network from input layer → hidden layers → output layer.

At each neuron:

Input → Weighted Sum + Bias → Activation Function → Output

It helps the network generate a prediction.



2. Cost Function

The cost function measures how different the model's prediction is from the actual output.

- Low cost → Better prediction
- High cost → Poor prediction
- The network learns by minimizing the cost function.

Mean Squared Error (MSE):

$$Cost = \frac{1}{n}\sum(\hat{y} - y)^2$$

Where:

$\hat{y}$ = predicted value

y = actual value

n = number of samples

Backward Propagation (Backpropagation)

Backpropagation is the process of sending the error backward through the neural network to improve its predictions.

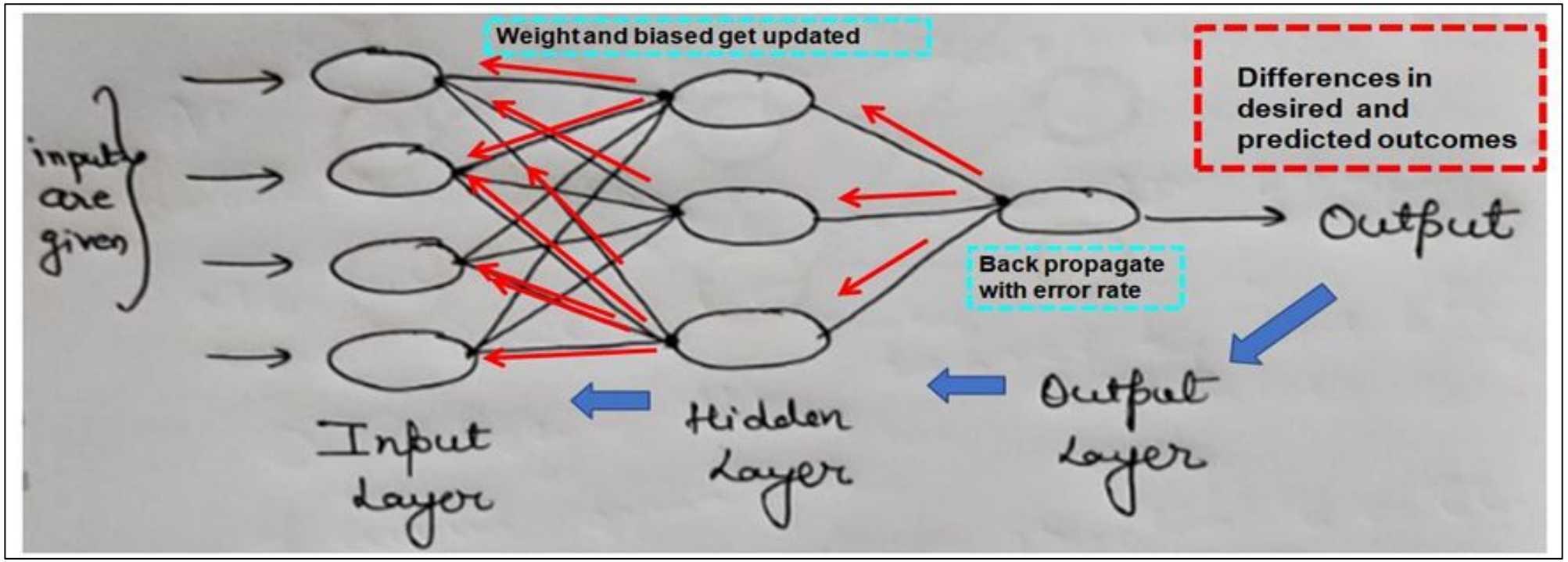
Calculate the error

→ Send error backward

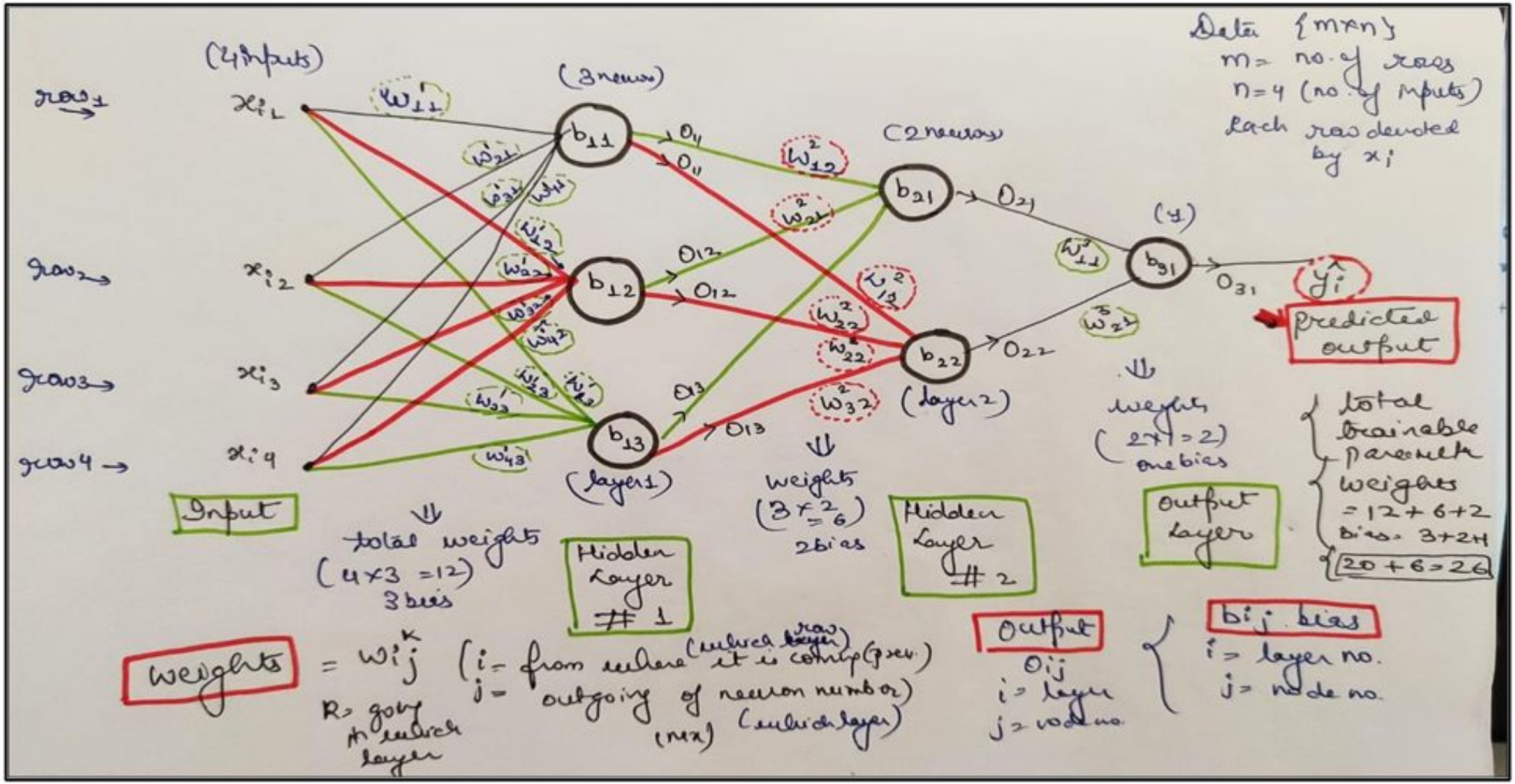
→ Update weights and biases

→ Make a better prediction next time

It means **Backpropagation = Learn from the error and adjust the network to make better predictions.**



Artificial Neural Networks (ANNs)



Neural Networks, Matrix Calculus & Backpropagation in NLP

Matrix Calculus in NLP

Matrix calculus provides the mathematical rules for calculating how the model's output changes when its weights and word vectors change.

For example:

$h=f(Wx+b)$

where W = weights, x = input word vectors, and b = bias.

Matrix calculus helps us calculate how a small change in weights or word vectors affects the model's output. This is important for training NLP models using backpropagation.

NLP Example: Sentiment Classification

Suppose the sentence is:

"This movie is good."

The word "**good**" is represented by a word vector:

$$x = [0.8,\ 0.6]$$

The neural network uses weights:

$$W = [0.5,\ 0.4]$$

and bias:

$$b = 0.1$$

Step 1: Calculate the output

$$z = Wx + b$$

$$z = (0.5 \times 0.8) + (0.4 \times 0.6) + 0.1$$

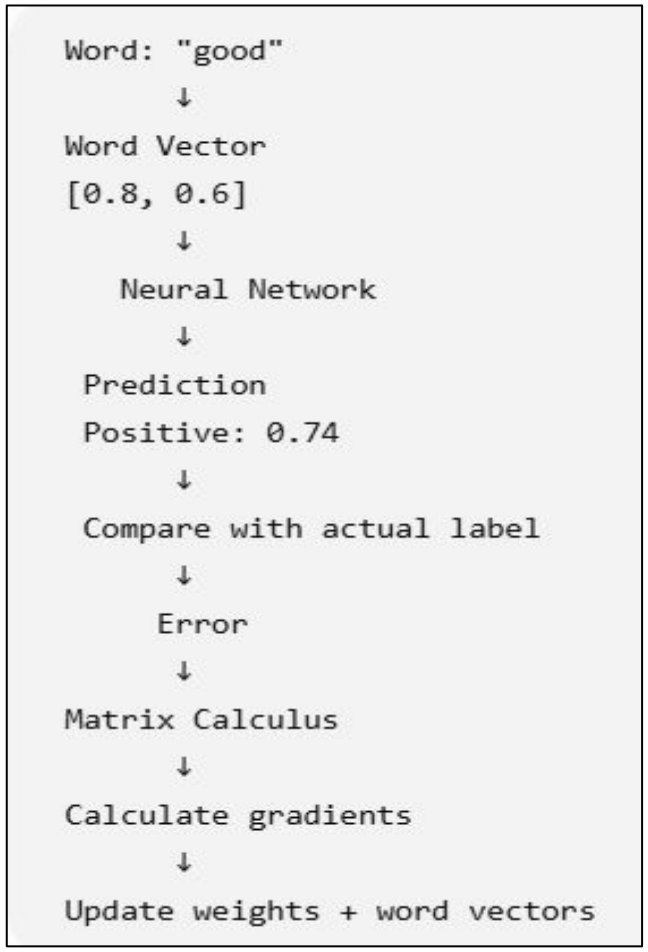
$$z = 0.74$$

After applying an activation function, the network produces a **Positive sentiment** prediction.

Step 2: Matrix Calculus

If the prediction is not correct, matrix calculus helps calculate:

How much should each weight and word-vector value change to reduce the error?



Matrix calculus tells the neural network:

"Which weights and word-vector values should be changed, and by how much, to make a better prediction?"

This is the mathematical foundation behind backpropagation and learning in NLP models.

Neural Networks, Matrix Calculus & Backpropagation in NLP

3. Backpropagation in NLP

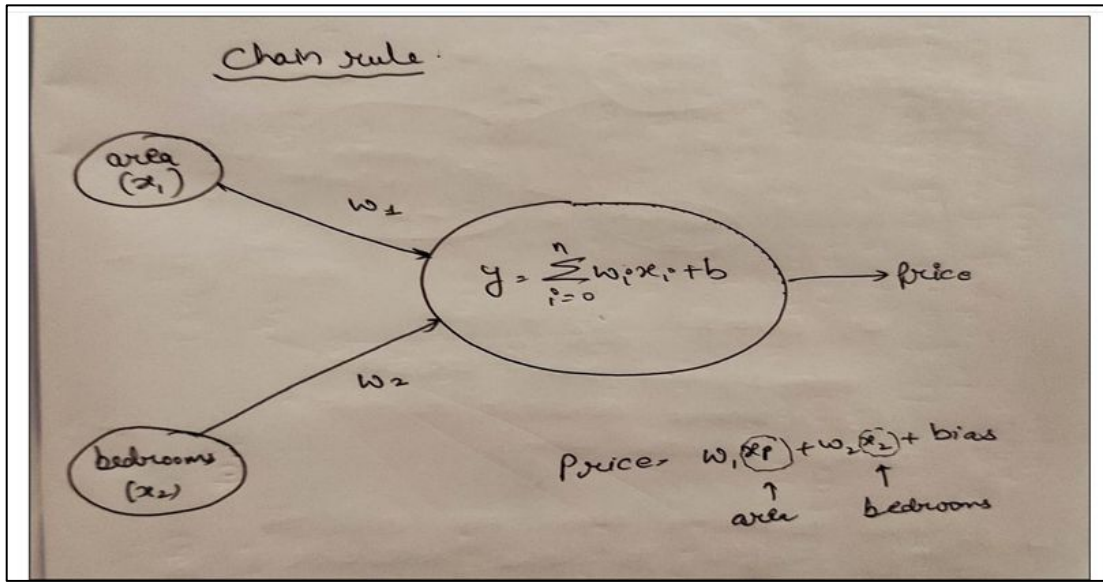
Backpropagation sends the prediction error backward through the network and calculates how each parameter should be changed to reduce the error.

Prediction
↓
Loss
↓
Backpropagation
↓
Gradients
↓
Update Weights + Word Vectors

Words → Vectors → Neural Network → Prediction → Loss → Backpropagation → Update Parameters → Better Prediction

Chain Rule in Neural Networks

The chain rule helps calculate how much each weight and bias contributes to the final error (loss) during backpropagation.



How it works

During forward propagation:

Input ⟶ Weighted Sum ⟶ Activation ⟶ Output ⟶ Loss

During backpropagation,

the chain rule works in the reverse direction:

Loss ⟶ Output Layer ⟶ Hidden Layer ⟶ Input Layer

It calculates gradients and helps update the weights and biases.

Example

If:

Input ⟶ Hidden Layer ⟶ Output ⟶ Loss

Then the chain rule calculates:

Then the chain rule calculates:

$$\frac{\partial Loss}{\partial Weight} = \frac{\partial Loss}{\partial Output} \times \frac{\partial Output}{\partial Hidden} \times \frac{\partial Hidden}{\partial Weight}$$

It means

Chain Rule = Break a complex calculation into smaller steps to find how each parameter affects the final error.

This allows backpropagation to update the network's weights and improve learning.

How word vectors flow through a neural network

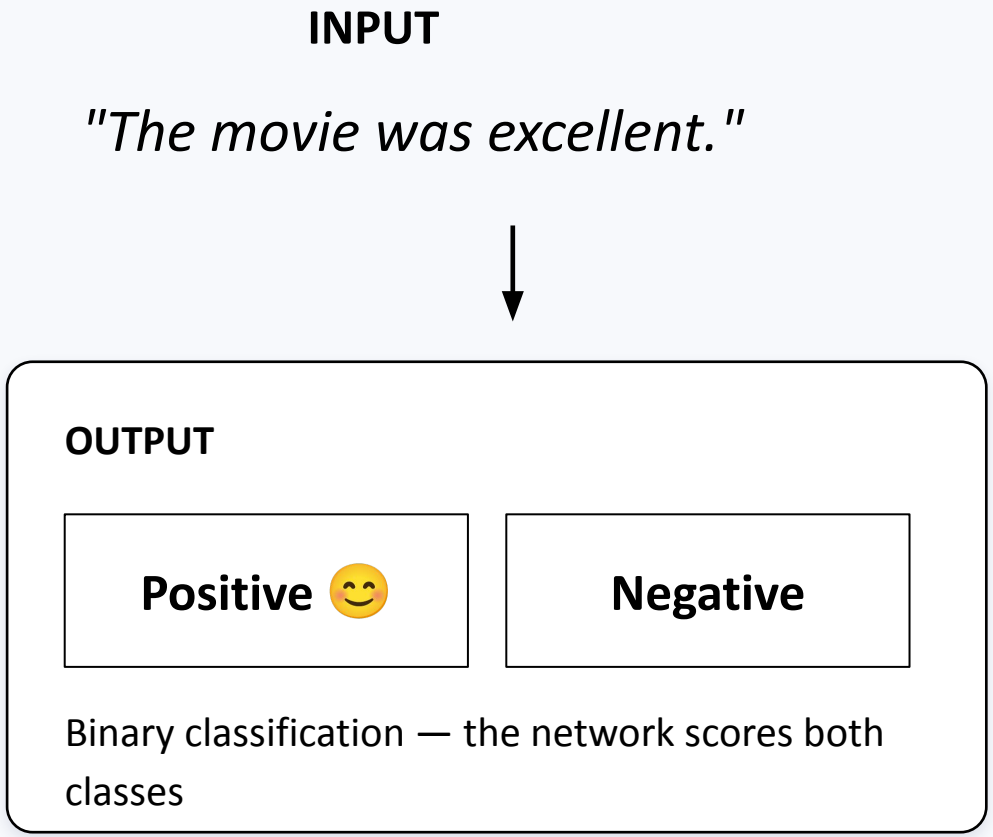
"The movie was excellent."



Classify sentiment from raw text

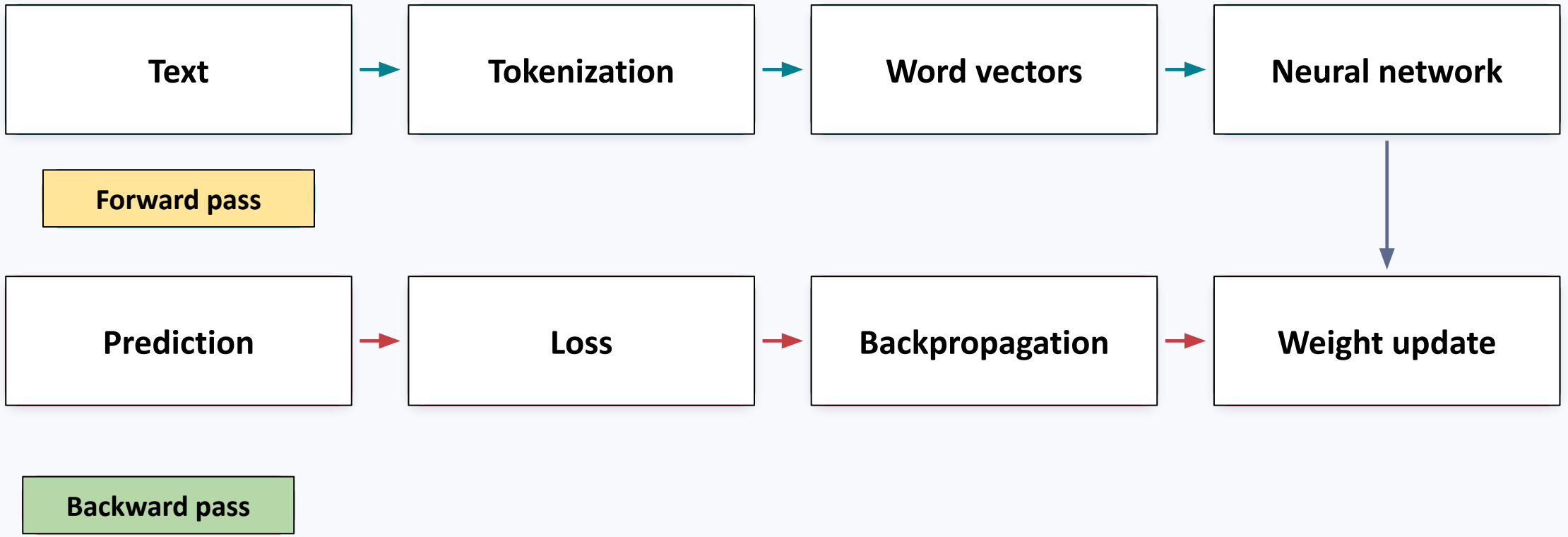
We want a neural network to decide whether a sentence expresses positive or negative sentiment.

- Neural networks only operate on numbers, not words
- So the sentence must first be converted into numeric form
- The network then produces a probability for each class
- The full journey: text → numbers → prediction



One pipeline, two directions

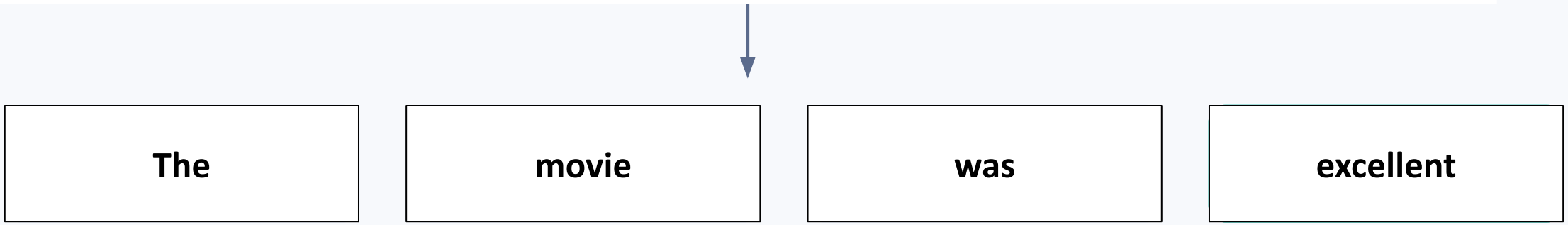
Forward propagation makes a prediction; backpropagation uses the error to improve the network.



STEP 1

Turning text into tokens

"The movie was excellent"



A neural network cannot perform math on the word “movie” — every token must become a number before it can go any further.

STEP 2

Word embeddings turn tokens into vectors

Each word is looked up in an embedding table (Word2Vec, GloVe, ...) and replaced with a vector of numbers.

Word	Vector (3-dim example)
The	[0.10, 0.20, 0.15]
movie	[0.40, 0.30, 0.25]
was	[0.20, 0.10, 0.30]
excellent	**[0.90, 0.80, 0.85]**

WHY 3 DIMENSIONS?

This example uses tiny 3-dimensional vectors so the arithmetic is easy to follow by hand.

IN PRACTICE

Real NLP models use 100, 300, or 768+ dimensions to capture much richer meaning.

STEP 3

Averaging into one sentence vector

Word	Vector (3-dim example)
The	[0.10, 0.20, 0.15]
movie	[0.40, 0.30, 0.25]
was	[0.20, 0.10, 0.30]
excellent	**[0.90, 0.80, 0.85]**

For this example, the sentence vector is simply the average of its four word vectors.

X = (v_The + v_movie + v_was + v_excellent) / 4

1st dim	(0.10 + 0.40 + 0.20 + 0.90) / 4	**= 0.40**
2nd dim	(0.20 + 0.30 + 0.10 + 0.80) / 4	**= 0.35**
3rd dim	(0.15 + 0.25 + 0.30 + 0.85) / 4	**= 0.3875**

SENTENCE VECTOR

X = [0.40, 0.35, 0.3875]

This single vector now represents the entire sentence — **ready to feed into the neural network.**

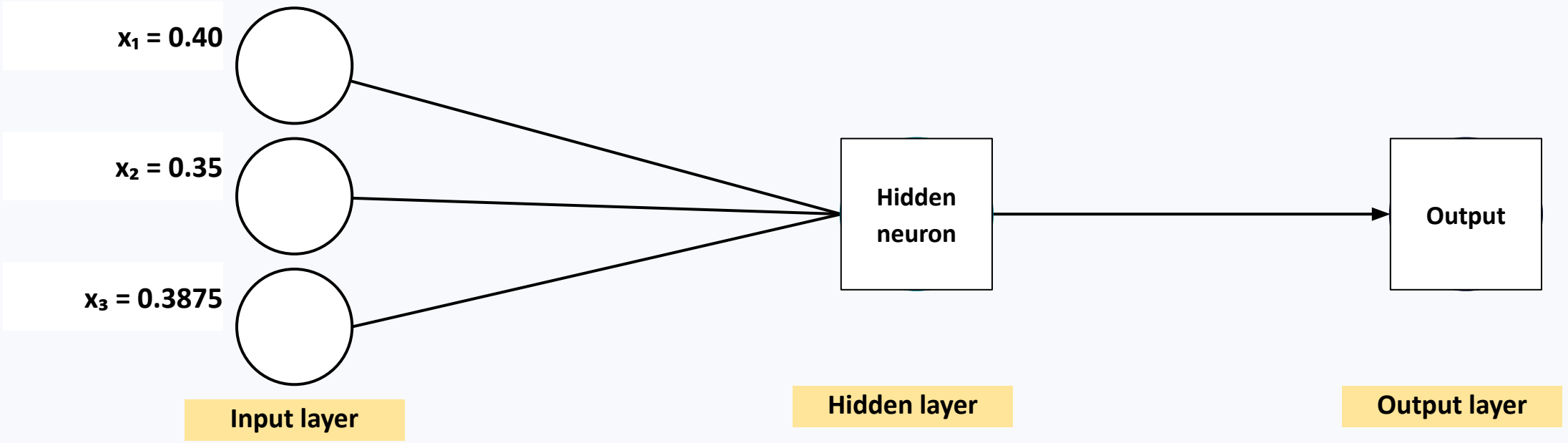
STEP 4

Feeding the network

The sentence vector becomes the input layer of a small neural network.

SENTENCE VECTOR

X = [0.40, 0.35, 0.3875]



Each connection carries a weight; the hidden neuron combines all three inputs into a single value.

STEP 5 and 6

Weighted sum, then activation

WEIGHTED SUM

$$Z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

w = [0.6, 0.8, 0.5] b = 0.1

Z = (0.6×0.40) + (0.8×0.35) + (0.5×0.3875) + 0.1

Z = 0.24 + 0.28 + 0.19375 + 0.1

Z = 0.81375

ACTIVATION — ReLU

ReLU(z) = max(0, z)

A = ReLU(0.81375) = 0.81375

Why activation functions?

Without them, a network is just a linear calculation. Activation functions introduce non-linearity, letting the model learn the complex patterns of language.

Suppose the hidden neuron has: W=[0.6,0.8,0.5] and bias: b=0.1

STEP 7–8

Output layer → sigmoid → prediction

OUTPUT LAYER

Now the hidden-layer output is passed to the output neuron.

A = ReLU(0.81375) = 0.81375

$Z_o = (A \times W_o) + b_o$

$W_o = 1.5 \quad b_o = 0.2$

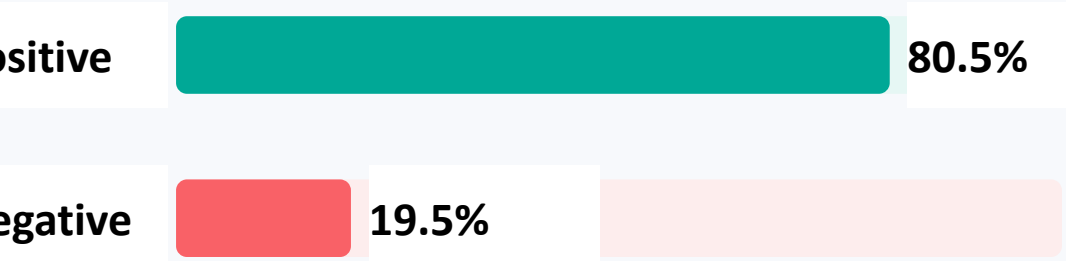
$Z_o = (0.81375 \times 1.5) + 0.2 = 1.420625$

SIGMOID FUNCTION

$\sigma(z) = 1 / (1 + e^{-z})$

$\sigma(1.420625) \approx 0.805$

PREDICTED PROBABILITIES



FINAL PREDICTION

Positive

The complete forward pass

Text	"The movie was excellent"
Tokens	The | movie | was | excellent
Word vectors	4 vectors, one per token
Sentence vector	X = [0.40, 0.35, 0.3875]
Weighted sum + ReLU	Z = 0.81375 $\rightarrow$ A = 0.81375
Output + sigmoid	$\sigma(1.420625) \approx 0.805$
Prediction	Positive — 80.5%

MEASURING ERROR

How wrong was the prediction? — loss

ACTUAL LABEL

Positive (y = 1)

PREDICTED

$\hat{y} = 0.805$

BINARY CROSS-ENTROPY LOSS

$$L = -[\, y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}) \,]$$

With $y = 1$ and $\hat{y} = 0.805$, the second term drops out:

$$L = -\log(0.805)$$

SIGMOID FUNCTION

$$\sigma(z) = 1 / (1 + e^{-z})$$

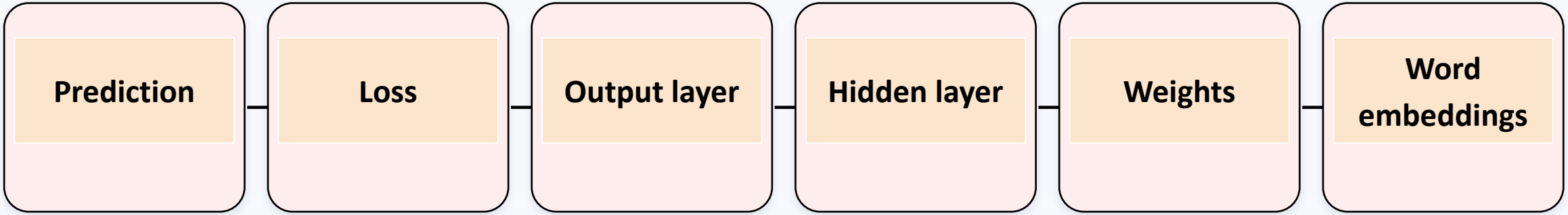
$$\sigma(1.420625) \approx 0.805$$

$L \approx 0.217$

Loss quantifies the gap between prediction and truth — the smaller, the better.

SENDING THE ERROR BACKWARD

Backpropagation



Error flows backward through the network, opposite to the forward pass.

THE CHAIN RULE

$$\partial L/\partial W = \partial L/\partial Output \times \partial Output/\partial Hidden \times \partial Hidden/\partial W$$

The loss depends on the output; the output depends on the hidden neuron; the hidden neuron depends on the weights. The chain rule links these together, so the network can work out exactly how much each weight contributed to the final error.

LEARNING FROM THE ERROR

Updating weights and word vectors

WEIGHT UPDATE RULE

$$W_{new} = W_{old} - \eta \cdot (\partial L / \partial W)$$

η = learning rate

Worked example

W_old = 0.6

gradient = 0.1

W_new = 0.599

WORD VECTORS UPDATE TOO

Embeddings aren't fixed — backpropagation nudges them as well, so the model learns better word representations over time.

“excellent” before training

[0.90, 0.80, 0.85]

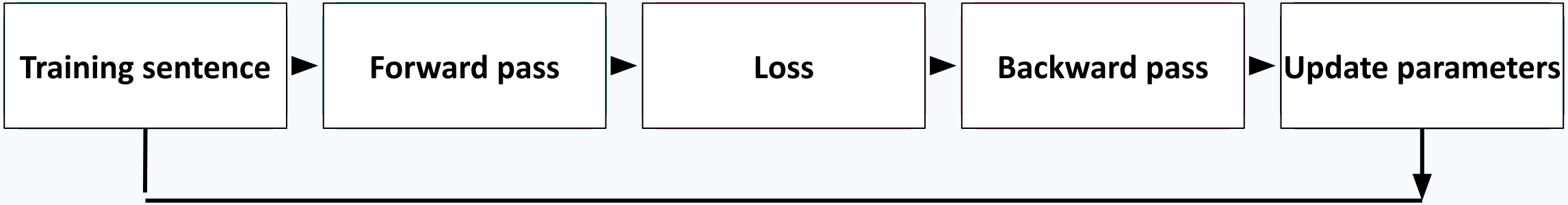
“excellent” after one update

[0.91, 0.81, 0.86]

TRAINING OVER TIME

The learning cycle repeats

One sentence is one lap. Training runs this loop for many sentences, over many epochs.



Across thousands of sentences, the network's weights and word embeddings gradually settle into values that make accurate predictions.

The same pattern powers many NLP tasks

Task	Input	Prediction
Sentiment analysis	Sentence	Positive / Negative
Spam detection	Email text	Spam / Not spam
Named entity recognition	Sentence	Person, location, organization
Text classification	Document	Category
Intent detection	User query	Intent
Language modeling	Previous words	Next word

NER EXAMPLE — “Rahul lives in Mumbai.”

Rahul	lives	in	Mumbai
PERSON	O	O	LOCATION

Negative Sampling

Optimization Technique in NLP

Negative Sampling

What is it?

- Negative Sampling is a technique used in Word2Vec to make training faster and more efficient.

Simple Meaning

- Normally, Word2Vec would compare a word with every word in the vocabulary to find the correct word. This is called full softmax, and it can be very expensive when there are millions of words.
- Negative Sampling avoids this. Instead of checking all words, it checks:
  - 1 correct word → positive example
  - A few incorrect/random words → negative examples

Example

Suppose the sentence is: "The cat drinks milk."

For the word "cat", Word2Vec may learn that "drinks" is a correct context word.

Word Pair	Type
cat → drinks	✓ Positive
cat → car	✗ Negative
cat → computer	✗ Negative
cat → mountain	✗ Negative

The model learns:

- cat + drinks → similar / related
- cat + car → not related
- cat + computer → not related

Instead of calculating probabilities for millions of words, Word2Vec only checks a small number of negative samples.

"Negative Sampling speeds up Word2Vec by training on a few incorrect words instead of calculating scores for the entire vocabulary.

THE PROBLEM

Softmax Doesn't Scale

Word2Vec learns which words appear near each other — but scoring every neighbor against the entire vocabulary is the bottleneck.

EXAMPLE SENTENCE

“The **dog chased** the **cat**”

The model must learn: “dog” relates to “chased”, and “dog” relates to “cat”.

For every single word pair, the model has to compute **a relationship score against** — every other word in the vocabulary before it knows which one is most likely.

REAL-WORLD VOCABULARY SIZE

1,000,000+

words the model must compare against — for every single training pair

That means **one softmax computation over** a million-way probability distribution — per word, per sentence, repeated for the entire training corpus.

Negative Sampling in Word2Vec

When we have a very large collection of text, such as millions or billions of words, Word2Vec has a problem.

The problem with a large vocabulary

Suppose our vocabulary contains **500,000** different words.

For every training example, the normal Word2Vec model using full softmax has to calculate the probability of all 500,000 words.

For example:

Target word = "king"
Context word = "queen"

The model has to compare "queen" with every word in the vocabulary:

king → queen
king → apple
king → car
king → computer
...
king → 500,000 words

Doing this again and again for millions of training examples is very slow and expensive.

2. How Negative Sampling helps

Negative sampling avoids checking every word.

Instead, we select only a few words.

For example:

- **1 positive pair →** `"king" – "queen"` ✅
- **5 negative pairs →** `"king" – "car"`, `"king" – "table"`, etc. ❌

The model now has to answer a simple question:

> **Is this a real word pair or a fake word pair?**

For example:

Word Pair	Label
king – queen	1 → Real
king – car	0 → Fake
king – table	0 → Fake
king – banana	0 → Fake
king – computer	0 → Fake

So instead of checking **500,000 words**, we may check only **6 pairs** (1 positive + 5 negative).

3. Why is this much faster?

Suppose:

Vocabulary = 500,000 words

With full softmax:

> Check ≈ **500,000 words** for every training step.

With negative sampling:

> Check only **1 + k words**

If `k = 5`:

> 1 positive + 5 negative = **6 words**

So the amount of work becomes much smaller.

Full Softmax:

`500,000 → very expensive`

Negative Sampling:

`6 → much faster`

This is why Word2Vec can be trained on **very large datasets**.

4. How are negative words selected?

The negative words are **not simply selected randomly with equal probability**.

Word2Vec uses the word frequency in the corpus.

A common formula is:

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_j f(w_j)^{3/4}}$$

Don't worry about the formula initially. The important idea is:

> **Frequently occurring words are more likely to be selected, but not too frequently.**

For example, words such as:

> **the, is, a, of**

occur very often.

If we selected negative words directly according to their frequency, these words would appear as negative samples **too often**.

The **3/4 power** reduces their dominance and gives less frequent words a better chance of being selected.

1. Why 6 words?

Suppose we choose:

- **1 positive word** = the actual context word
- **5 negative words** = randomly selected incorrect words

Then:

$$1 + 5 = 6$$

For example:

Target word = **king**

Actual context word:

> king → queen ✅ **Positive**

Five negative samples:

> king → car ❌
> king → apple ❌
> king → table ❌
> king → computer ❌
> king → mountain ❌

So the model trains on **6 word pairs**.

But it could be different:

- `k = 2` → 1 + 2 = **3 pairs**
- `k = 5` → 1 + 5 = **6 pairs**
- `k = 10` → 1 + 10 = **11 pairs**
- `k = 20` → 1 + 20 = **21 pairs**

So **6 is not fixed**. It depends on the value of **k**.

THE CORE IDEA

Instead of ranking a target word against the whole vocabulary, Negative Sampling reframes training as a handful of simple yes/no decisions.

THE OLD WAY

Full Softmax

"Which of ALL 1,000,000 words is the true neighbor?"

- One giant multi-class classification
- Recomputes a score for every vocabulary word
- Cost grows directly with vocabulary size

THE NEGATIVE-SAMPLING WAY

A Few Small Checks

"Is THIS pair real? Is THAT pair fake?" — asked a few times.

- A handful of small binary classifications
- 1 real (positive) pair + a few random fake (negative) pairs
- Cost stays nearly constant, however big the vocabulary gets

The Training Objective

For each positive pair, **Negative Sampling maximizes this objective** means it just rewards **separating real pairs from fake ones.**

$$\log \sigma(v_{w_o} \cdot v_{w_i}) + \sum_{k=1..K} \mathbb{E}_{w \sim Pn(w)} [\log \sigma(-v_{w} \cdot v_{w_i})]$$

σ = sigmoid function · v_{ui} = context/target word vectors · K = number of negative samples · Pn(w) = noise distribution words are drawn from

Positive term

$$\log \sigma(v_wo \cdot v_wi)$$

Pulls the target *v_wo* and its true context word *v_wi* closer together in vector space — pushing this score toward 1.

Negative term

$$\Sigma \log \sigma(-v_wk \cdot v_wi)$$

Pushes the target apart from **K randomly sampled words** — pushing each of those scores toward 0.

EXAMPLE

Vocabulary & Training Data

A tiny 5-word vocabulary makes the mechanics easy to see — the same logic runs at millions-of-words scale.

STEP 1 — VOCABULARY (5 WORDS)

dog	cat	chased	apple	banana

STEP 2 — EXTRACTING PAIRS FROM A SENTENCE

"The **dog chased** the **cat**"

TARGET WORD		CONTEXT WORD	CONTEXT WORD
dog	→	**chased**	**cat**

Positive context words for "dog": chased, cat — these become the model's positive training pairs.

STEP 3

Positive Pair + Negative Samples

Rather than scoring “dog” against all 5 vocabulary words, we keep 1 real pair and randomly draw 2 fake ones.

POSITIVE PAIR — keep as-is from the sentence

“dog” ↔ “chased” **Label: 1 (related)**

NEGATIVE PAIRS — randomly sampled unrelated words

“dog” ↔ “apple” **Label: 0 (unrelated)**

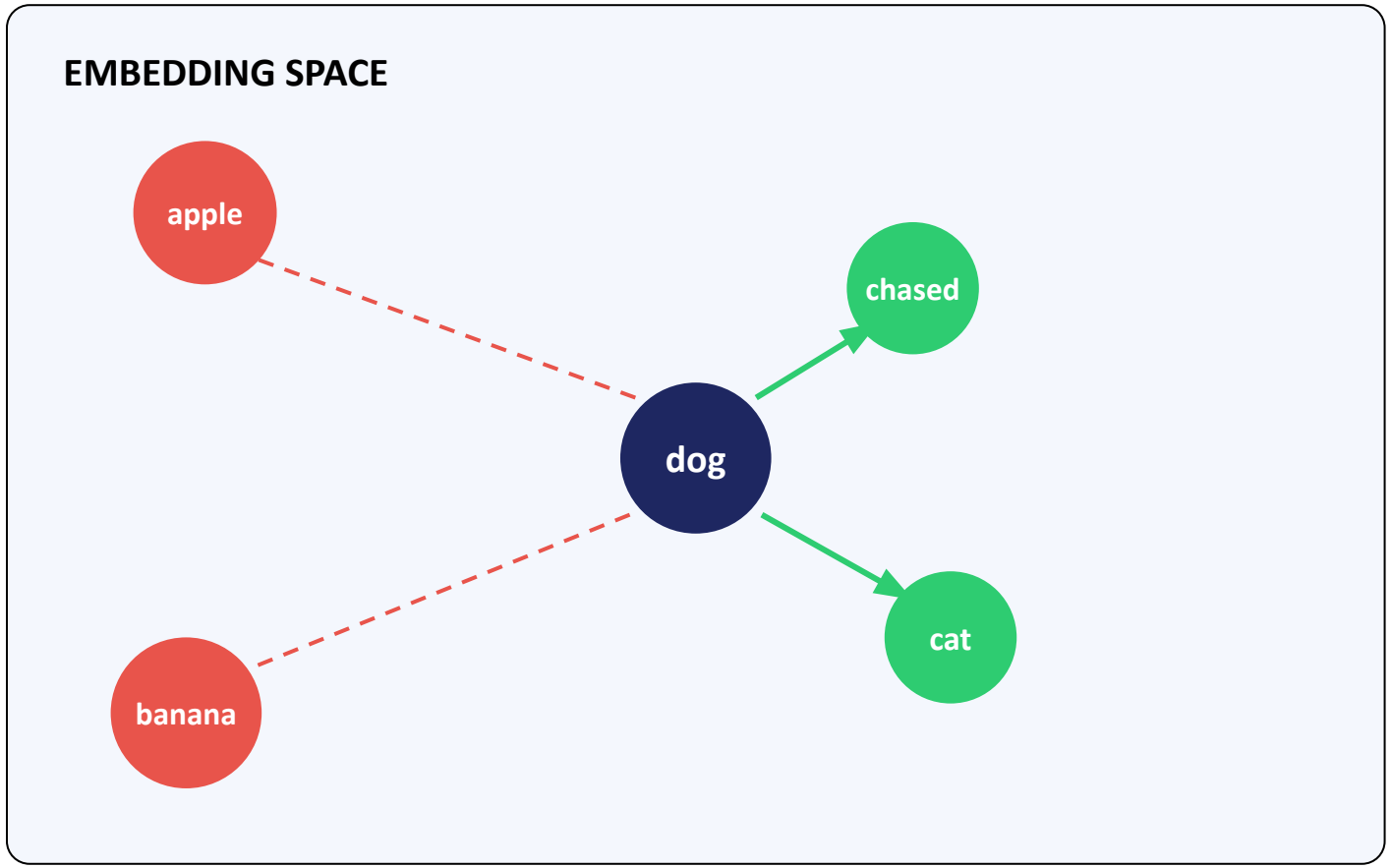
“dog” ↔ “banana” **Label: 0 (unrelated)**

Result: only 3 pairs to process for “dog” — not all 5 vocabulary words.

STEP 4–5

Training: Pull Together, Push Apart

Each pair nudges word vectors in embedding space — positive pairs move closer, and negative pairs move apart.



PULL TOGETHER

Positive pair (dog, chased)

The model increases the similarity between their vectors ,closer in embedding space.

PUSH APART

Negative pairs (dog, apple / banana)

The model decreases their similarity — so unrelated words further apart.

How Much Time Does This Save?

Every training pair only touches a handful of words instead of the entire vocabulary

The model processes a **near-constant handful of** words per pair — **not the full vocabulary** — no matter how large the vocabulary grows.

REAL-WORLD SCALE (1M-WORD VOCABULARY)

1,000,000 words scored (full softmax)

vs

5–20 words scored (negative sampling)

KEY POINT

Training cost becomes roughly independent of vocabulary size — which is exactly what makes Word2Vec practical on real corpora.

LINGUISTIC STRUCTURE IN NLP

the way a language is organized and how words are arranged to form meaningful sentences.

Dependency Parsing

Dependency Parsing is a technique in NLP that helps a computer understand **how words in a sentence are connected grammatically.**

It identifies:

- Who is doing the action? → **Subject**
- What is the action? → **Verb**
- What is affected by the action? → **Object**

Example

Sentence:

She eats an apple.

Dependency relationships:

She → eats → apple

- She → Subject (who is doing the action?)
- eats → Main verb (what is the action?)
- apple → Object (what is being eaten?)
- an → Determiner that describes apple

So, the computer understands:

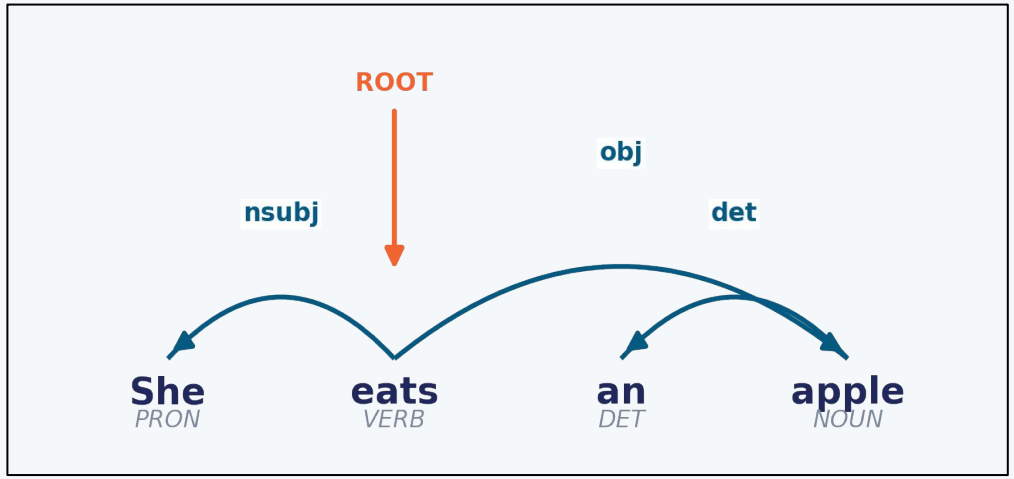
She is doing the action eats, and apple is receiving the action.

Dependency Parsing maps each word to the word it grammatically depends on, helping machines understand the structure and meaning of a sentence.

Main Use: It helps NLP systems understand "who did what to whom."

Understanding Dependency Parsing?

- Dependency parsing is how an NLP system figures out the **grammatical skeleton** of a sentence
  - not by grouping words into phrases,
  - but by drawing a direct link from every word to the single other word it depends on.



- Each link is called an **arc**, it runs from a **head** to its dependent
- and it carries a label naming the grammatical relation — subject, object, modifier, and so on.
- **The result is a tree,** rooted at the sentence's main verb, that tells a machine exactly **who is doing what to whom.**

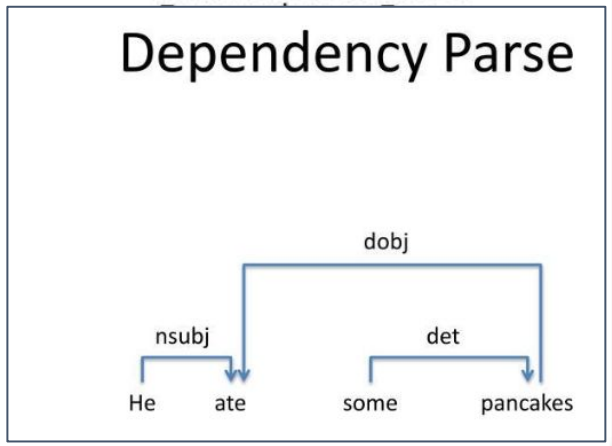
"She eats an apple." — eats is the ROOT; She and apple both depend on it.

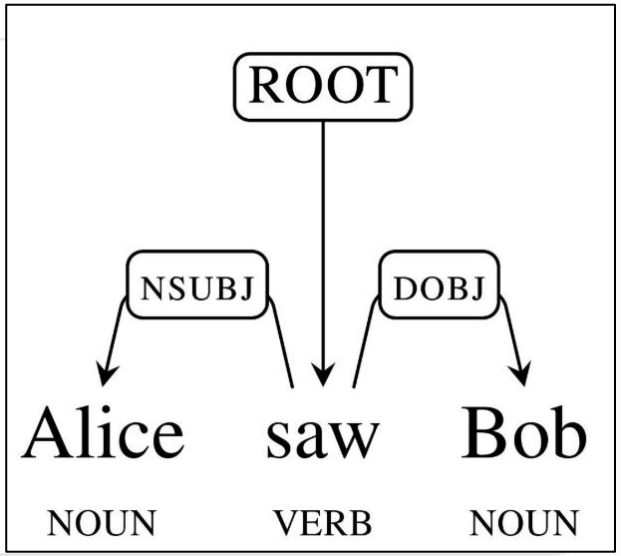
no head, no meaning. A word's role only makes sense in relation to the word that governs it.

Five Building Blocks of a Dependency Tree

Head	Dependent	Root	Relation Label
The word that governs another; it determines the grammatical category of the phrase (e.g. the verb governs the sentence).	The word that relies on the head for its role — a subject, an object, a modifier.	The one word with no head of its own — almost always the sentence's main verb. Every tree has exactly one.	The tag on each arc naming the grammatical function — nsubj, obj, amod, and more.

Sentence: *The boy eats an apple.*



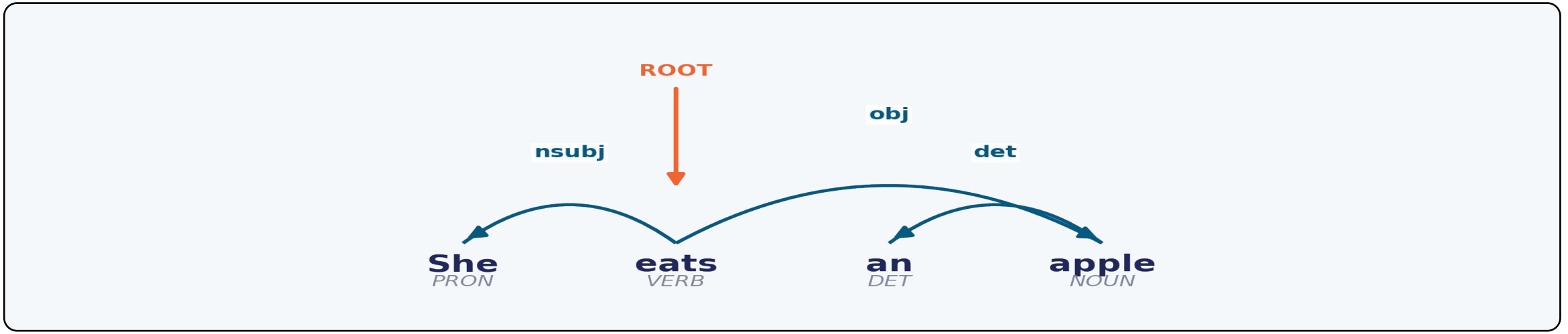


- **eats** → **Root**
- **boy** → depends on *eats* as the **subject (nsubj)**
- **apple** → depends on *eats* as the **object (obj)**
- **The** → depends on *boy* as a **determiner (det)**
- **an** → depends on *apple* as a **determiner (det)**

So, a dependency tree tells us **which word depends on which other word and what grammatic relationship connects them**.

Building Block	Example	Explanation
Root	**eats**	Main verb; it has no head.
Head	**eats**	Governs other words in the sentence.
Dependent	**boy**, **apple**	These words depend on **eats** for their grammatical role.
Relation Label	**nsubj**, **obj**, **det**	Shows the relationship between words.

Reading a Dependency Tree, Word by Word

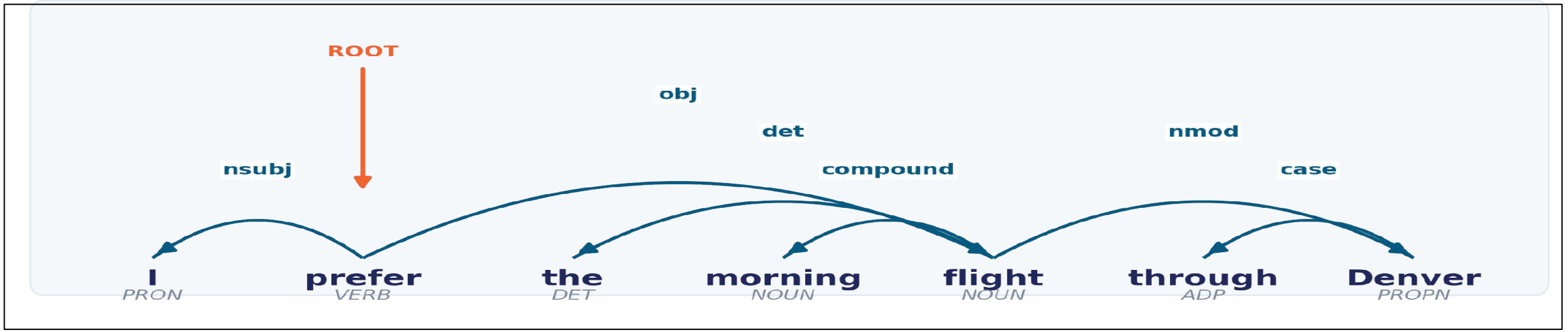


Word	Head	Relation	What it means
She	**eats**	**nsubj**	The nominal subject — who performs the action.
eats	**ROOT**	**root**	The main verb; the tree's anchor point.
an	**apple**	**det**	A determiner fixing which/how many apples.
apple	**eats**	**obj**	The direct object — what receives the action.

EXAMPLE

A Richer Sentence: Nested Modifiers

“I prefer the morning flight through Denver.”



Notice how flight — not prefer — becomes the local head for everything that describes it:

det

the → flight

marks definiteness

compound

morning → flight

noun modifying a noun

nmod

Denver → flight

nominal modifier

case

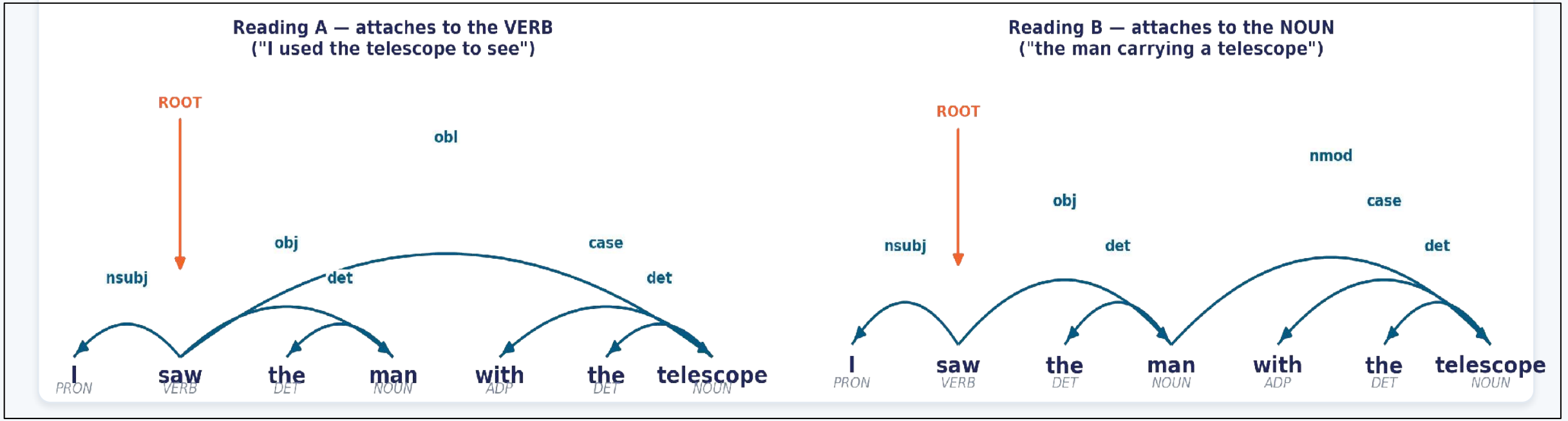
through → Denver

preposition marks the case

WHY STRUCTURE MATTERS

Resolving Ambiguity: PP-Attachment

“I saw the man with the telescope" is famously ambiguous — and the two meanings are literally two different dependency trees. The parser's job is to decide which one the sentence intends.



obl vs. nmod is the whole difference: attach the prepositional phrase to the verb (saw) and the telescope is the instrument; attach it to the noun (man) and the telescope belongs to the man. Real parsers use context, word meaning, and probability to pick the right one.

The Relation Tag Set: Universal Dependencies

Universal Dependencies (UD) defines ~37 standard relation labels shared across languages. These **twelve** cover the large majority of everyday sentences:

nsubj **nominal subject** — who performs the action	**obj** **direct object** — what receives the action	**iobj** **indirect object** — the recipient/beneficiary
root **root of the sentence** — the main predicate, no head	**det** **determiner** — articles: the, a, this, some	**amod** **adjectival modifier** — an adjective describing a noun
advmod **adverbial modifier** — an adverb modifying a verb/adj	**nmod** **nominal modifier** — a noun modifying another noun	**case** **case marking** — a preposition/postposition
aux **auxiliary** — helper verbs: is, has, will	**cc / conj** **coordination** — and, or, but joining items	**punct** **punctuation** — commas, periods, quotes

Extracting Dependencies in Python

dependency_parse.py

```
import spacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("I prefer the morning flight through Denver.")

for token in doc:
    print(f"{token.text:<10} "
          f"head={token.head.text:<10} "
          f"dep={token.dep_:<8} "
          f"pos={token.pos_}")
```

Output

```
I          head=prefer     dep=nsubj    pos=PRON
prefer     head=prefer     dep=ROOT     pos=VERB
flight     head=prefer     dep=obj      pos=NOUN
Denver     head=flight     dep=nmod     pos=PROPN
```

For example:

I ──nsubj──→ prefer
prefer ──ROOT
flight ──obj──→ prefer
Denver ──nmod──→ flight

So, **DEP = the grammatical relationship between a word and its HEAD word.**

1 Load a pipeline

spacy.load("en_core_web_sm") loads a pretrained tokenizer, tagger, and dependency parser together.

2 Run it on text

Calling nlp(text) tokenizes and parses in one pass, returning a Doc of linked Token objects.

3 Read the tree off each token

token.head gives the governing word, token.dep_ gives the relation label, and token.pos_ gives its part of speech.

NLTK has no built-in neural dependency parser — it wraps external ones (Stanford CoreNLP) or works with phrase-structure trees instead.

Dependency parsing is used in:

- **Information Extraction**: Finds who did what to whom.
- **Chatbots:** Understands user intent and key information.
- **Question Answering:** Matches questions with relevant answers.
- **Grammar Checking**: Detects grammar and modifier errors.
- **Machine Translation:** Preserves grammatical relationships across languages.
- **Sentiment Analysis:** Connects opinions to the correct entities.

Thank You